TEMA 3

ORGANIZACIÓN APLICACIÓN

Desarrollo Interfaces



IES HENRI MATISSE

IES HENRI MATISSE

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

TEMA 3. ORGANIZACIÓN APLICACION

Contenido

1. Introducción....3
2. Organización ventana....3
2.1. Barra de menú....5
2.1.1. Menu Item....6
2.2. Barra de herramientas....9
2.3. Ribbon....11
2.3.1. Microsoft....11
2.3.2. Fluent Ribbon....17
2.4. Tipo desplegable o acordeón....20
2.4.1. Expander....22
2.5. Pestañas....24
2.6. Hamburguer menú....25
3. Templates....28
3.1. Control Template....29
3.2. Data Template....31
ANEXO I. INSTALACIÓN CONTROLES....32
ANEXO II. MAHAPPS....35
ANEXO III. Recursos....40
Bibliografía....42

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

1. Introducción

En este tema veremos algunos de los principales componentes que se pueden utilizar en el diseño de interfaces y más concretamente con WPF dentro de .NET

En lugar de realizar una lista pormenorizada de los componentes haré un listado de los mismos según su funcionalidad.

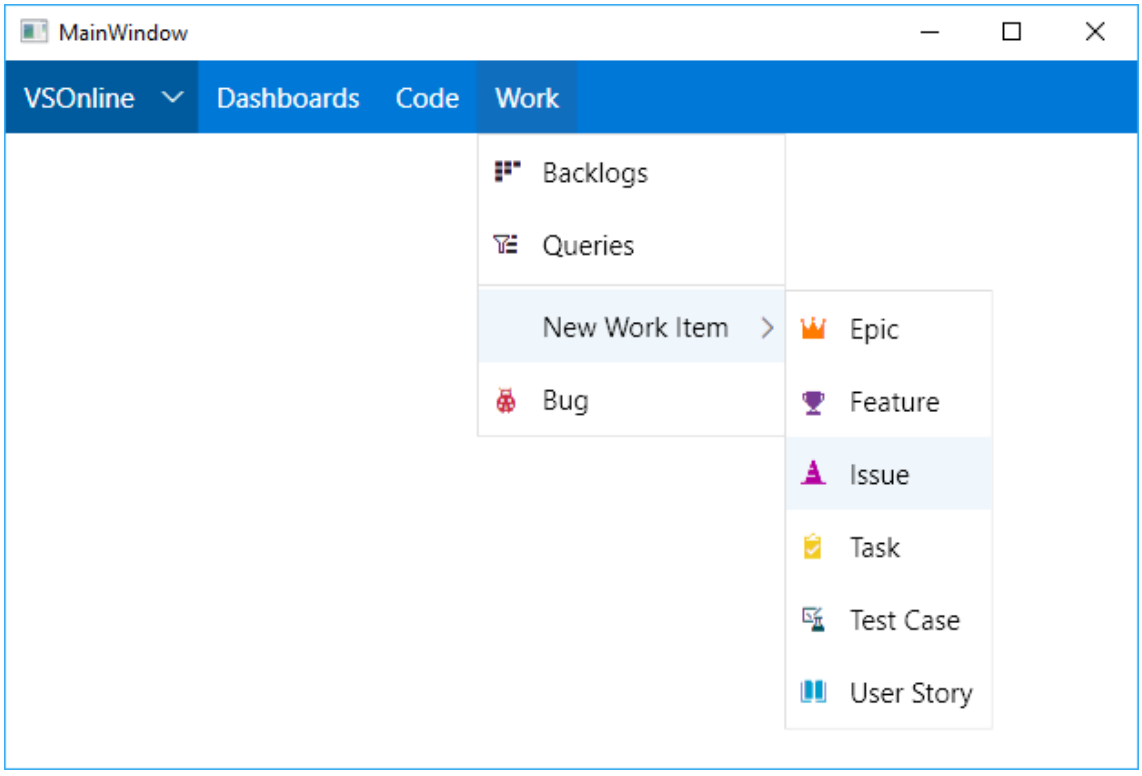
Ya digo, existen muchos, a lo cual hay que añadir los que terceros desarrollan de forma independiente, ya sean libres o de pago.

En este tema hablaremos de los **elementos utilizados para organizar una interfaz principal**.

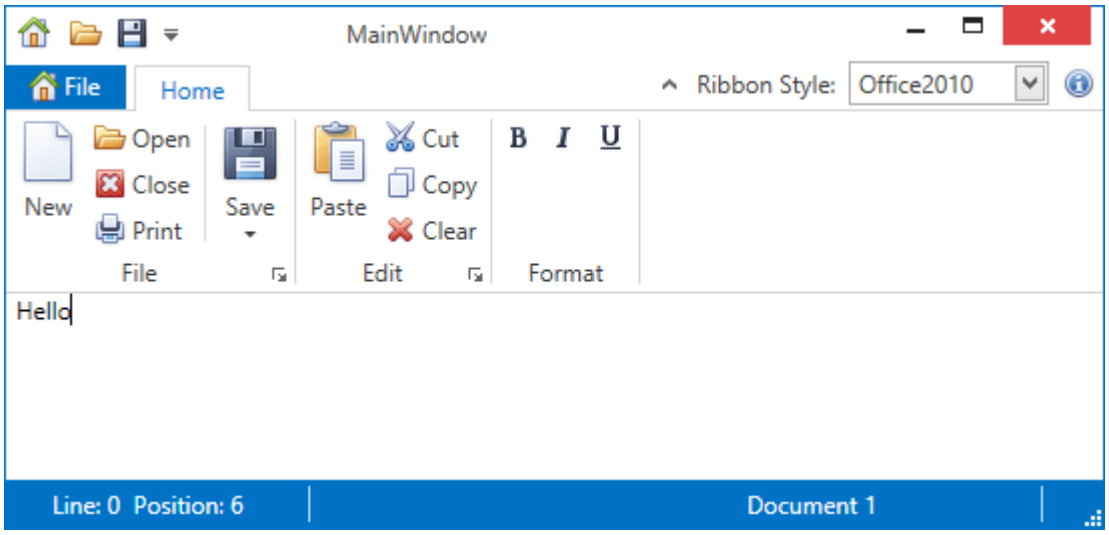
2. Organización ventana

En este punto trataremos de los elementos que suelen construir una interfaz principal para organizar todas las opciones y posibilidades que pueda tener.

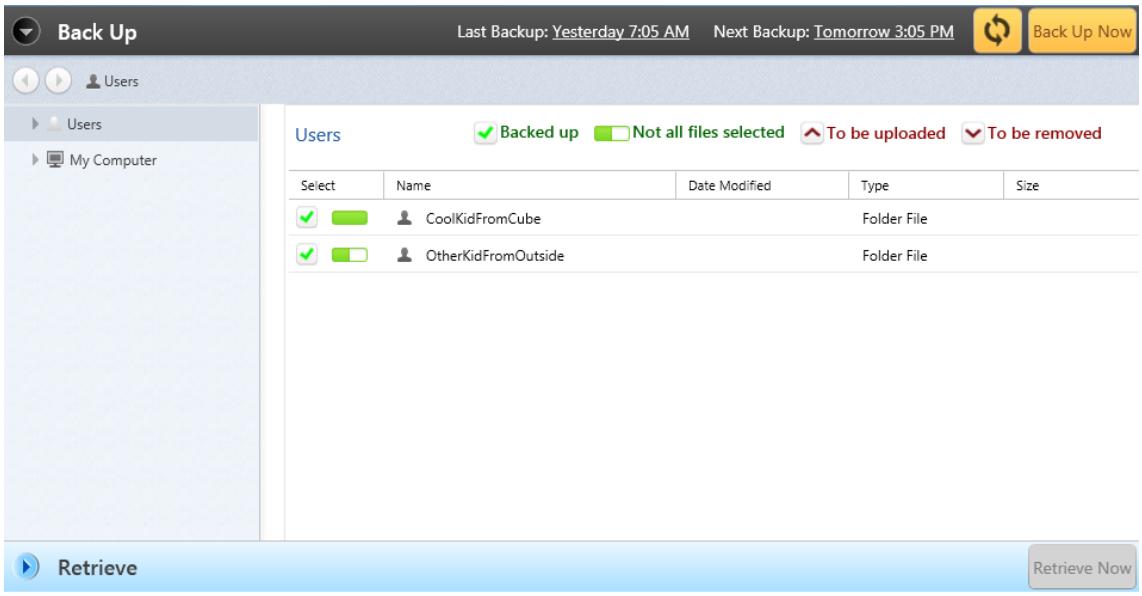
Analizaremos las principales, aunque siempre puede haber variaciones de un framework a otro. Todas ellas pueden combinarse y de esta manera conseguir una interfaz más rica y flexible.



Estos componentes pueden seguir un enfoque más clásico como una **barra de menús** que tiene asociada una **barra de herramientas**, o pueden tener un enfoque más moderno como la famosa cinta o ribbon de Microsoft.



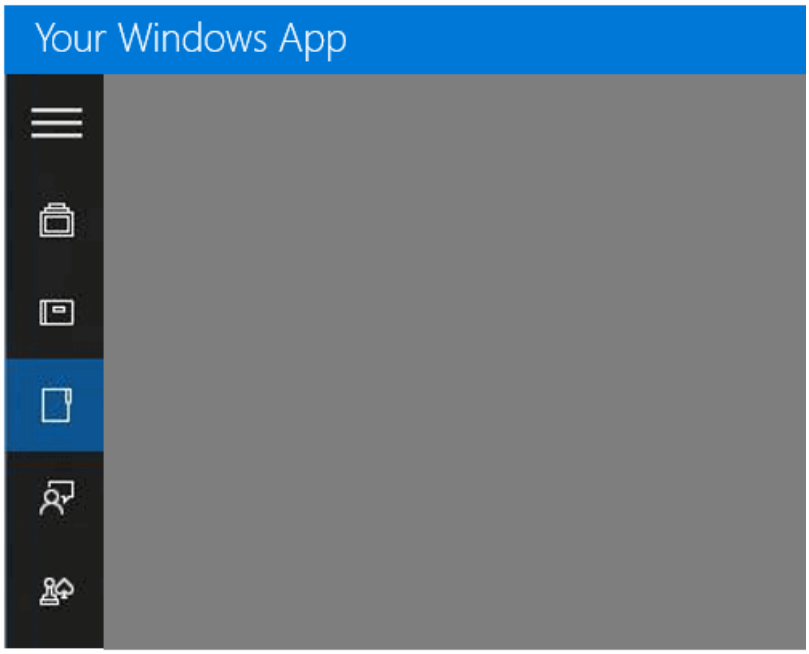
También podemos orientar nuestra aplicación a un estilo más parecido a una web, con un menú o un elemento acordeón en el cual albergar las principales opciones.



Otra posibilidad consistiría en disponer de una interfaz principal con unos **botones que nos lleven a las distintas opciones del programa**. Este tipo de interfaces son útiles en caso de utilizar **pantallas táctiles** pues facilitan la interactuación con el usuario, sin embargo son algo engorrosas para interfaces sin dispositivo táctil puesto que pierdes la perspectiva del conjunto de la aplicación. Sobre todo si existen multitud de opciones.

Otra opción que se aplica mucho en aplicaciones móviles es la implementación de un ***Hamburguer Menu*** que dispone de una barra de opciones que muestra simplemente los botones principales, pero que luego podemos desplegar para ver el total de las opciones de las que dispone el programa.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



Para complementar las anteriores siempre suele utilizarse una distribución del contenido de la aplicación mediante pestañas, aunque, como ya he dicho para **complementar**, puesto que **no es nada recomendable** realizar una interfaz solamente con pestañas, puesto que en caso de tener muchas opciones el programa se vuelve inmanejable.

2.1. Barra de menú

Este control se utiliza para la organización de las opciones de las que dispone un programa, se ubica en la parte superior y divide las funcionalidades en secciones, de las cuales cuelgan las subfuncionalidades.

Este tipo de componentes estructuran de forma jerárquica las distintas opciones de las que dispone un programa.

Con este control se suelen utilizar propiedades generales para su configuración como por ejemplo: **Background, Effect, Height, Width o ToolTip**.

Hay otras propiedades que suelen utilizarse con este control y que son más particulares del mismo. Como por ejemplo:

- **Items**: colección donde se guardan los ítems del menú.
- **IsMainMenu**: determina si el menú es el principal o no.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Este es el control principal que actúa como contenedor de las opciones que cuelgan del menú.

2.1.1. Menu Item

El anterior componente se muestra vacío ya que se trata de un contenedor simplemente. Para añadir opciones debemos utilizar la clase MenuItem, a través de la cual podremos especificar las opciones de manera jerárquica. Por ejemplo, si definimos un MenuItem y añadimos varias opciones dentro de él se estructurará de forma jerárquica y éstos pasarán a ser sub ítems del principal.

```
<Menu x:Name="menuPrueba" HorizontalAlignment="Left" Height="Auto"
VerticalAlignment="Top" Width="518" IsMainMenu="True">
            <MenuItem Header="Fichero">
                <MenuItem Header="Nuevo" />
                <MenuItem Header="Abrir" />
                <MenuItem Header="Guardar" />
                <Separator />
                <MenuItem Header="Salir" />
            </MenuItem>
            <MenuItem Header="Ayuda">
                <MenuItem Header="Acerca de"/>
            </MenuItem>
        </Menu>
```

Fijaos que nos quedan huecos a la izquierda de los nombres de los ítems, éstos se dejan para añadir otras componentes como iconos o checkboxes.

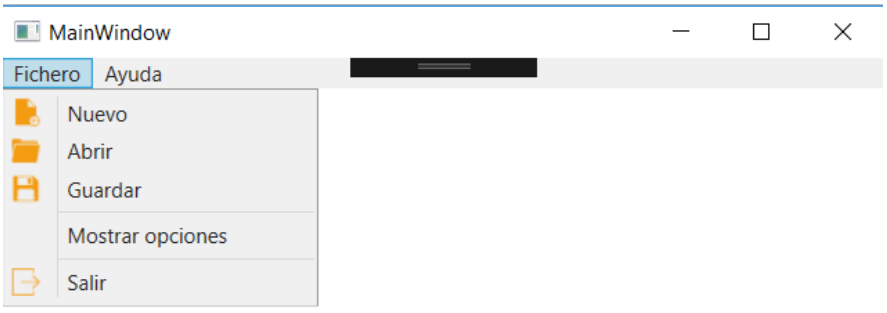
También se utiliza el control **Separator** para organizar las distintas opciones.

Para añadir iconos se realiza de la siguiente forma:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<Menu x:Name="menuPrueba" HorizontalAlignment="Left" Height="Auto"
VerticalAlignment="Top" Width="518" IsMainMenu="True">
            <MenuItem Header="Fichero">
                <MenuItem Header="Nuevo" >
                    <MenuItem.Icon>
                        <Image Source="icons/Nuevo.png"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="Abrir" >
                    <MenuItem.Icon>
                        <Image Source="icons/Abrir.png"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="Guardar" >
                    <MenuItem.Icon>
                        <Image Source="icons/Guardar.png"/>
                    </MenuItem.Icon>
                </MenuItem>
                <Separator />
                <MenuItem Header="Mostrar opciones" IsCheckable="True"/>
                <Separator />
                <MenuItem Header="Salir" >
                    <MenuItem.Icon>
                        <Image Source="icons/Salir.png"/>
                    </MenuItem.Icon>
                </MenuItem>
            </MenuItem>
            <MenuItem Header="Ayuda">
                <MenuItem Header="Acerca de"/>
            </MenuItem>
        </Menu>
```

En la figura siguiente se puede apreciar el resultado



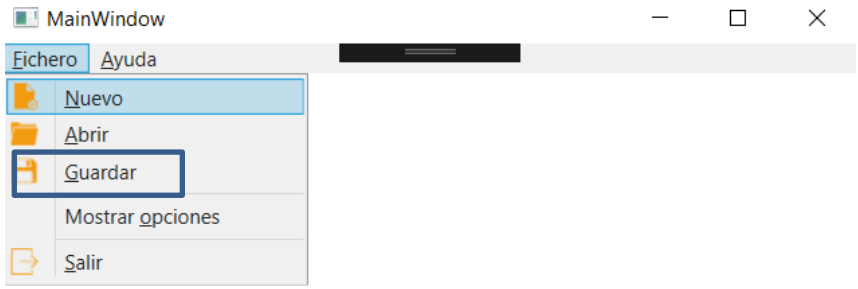
C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Para activar las opciones de un menú podemos utilizar el ratón o bien el teclado. Para subrayar la tecla que utilizaremos para acceder a la opción del menú mediante el teclado colocaremos el guion bajo antes de la letra que queremos que sea la seleccionada.

Por ejemplo, modificad el ejemplo anterior de la siguiente manera:

```
<Menu x:Name="menuPrueba" HorizontalAlignment="Left" Height="Auto"
VerticalAlignment="Top" Width="518" IsMainMenu="True">
            <MenuItem Header="_Fichero">
                <MenuItem Header="_Nuevo" >
                    <MenuItem.Icon>
                        <Image Source="icons/Nuevo.png"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="_Abrir" >
                    <MenuItem.Icon>
                        <Image Source="icons/Abrir.png"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="_Guardar" >
                    <MenuItem.Icon>
                        <Image Source="icons/Guardar.png"/>
                    </MenuItem.Icon>
                </MenuItem>
                <Separator />
                <MenuItem Header="_Mostrar opciones" IsCheckable="True"/>
                <Separator />
                <MenuItem Header="_Salir" >
                    <MenuItem.Icon>
                        <Image Source="icons/Salir.png"/>
                    </MenuItem.Icon>
                </MenuItem>
            </MenuItem>
            <MenuItem Header="A_yuda">
                <MenuItem Header="Acerca _de"/>
            </MenuItem>
        </Menu>
```

En la figura siguiente se observa el resultado:



También podemos activar combinaciones de teclas para acceder a la opción correspondiente de forma más rápida. Por ejemplo, para abrir un fichero más rápidamente

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

podemos activar como atajo de teclado la combinación de **Ctrl + A**. Este tipo de atajos de teclado lo estudiaremos más adelante cuando veamos **Comandos** en WPF.

2.2. Barra de herramientas

Este control se asemeja a un panel con botones, aunque tiene algunas funcionalidades que lo distinguen, como por ejemplo la posibilidad de añadir separadores de forma sencilla, reubicar las distintas barras de herramientas, o incluso mostrar de forma solapada aquellos componentes que no se visualizan en la barra de herramientas por tener una anchura demasiado pequeña.

Este control está asociado a **ToolBarTray** que nos sirve de contenedor en caso de disponer de varias barras de herramientas. En caso de utilizar una solamente podemos dejarla en solitario sin el control **ToolBarTray**.

Se definen las etiquetas de ToolBar y luego se añaden los botones y separadores.

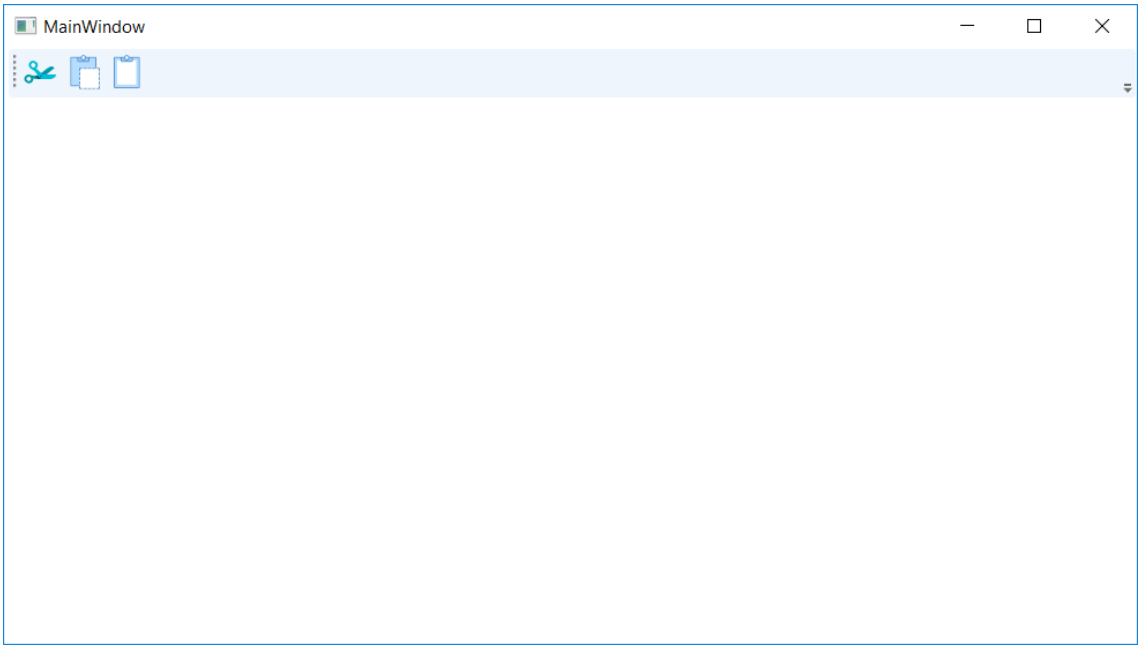
```
<DockPanel>
  <ToolBar DockPanel.Dock="Top">
      <Button Command="Cut" ToolTip="Cut selection to Windows Clipboard.">
          <Image Source="/Images/cut.png" Height="24" />
      </Button>
      <Button Command="Copy" ToolTip="Copy to Windows Clipboard.">
          <Image Source="/Images/copy.png" Height="24"/>
      </Button>
      <Button Command="Paste" ToolTip="Paste from Windows Clipboard.">
        <StackPanel >
            <Image Source="/Images/paste.png" Height="24" />
            <TextBlock Margin="3,0,0,0">Paste</TextBlock>
        </StackPanel>
       </Button>
   </ToolBar>
   <Grid>

   </Grid>
</DockPanel>
```

Este es el resultado que obtendríamos:

IES HENRI MATISSE

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

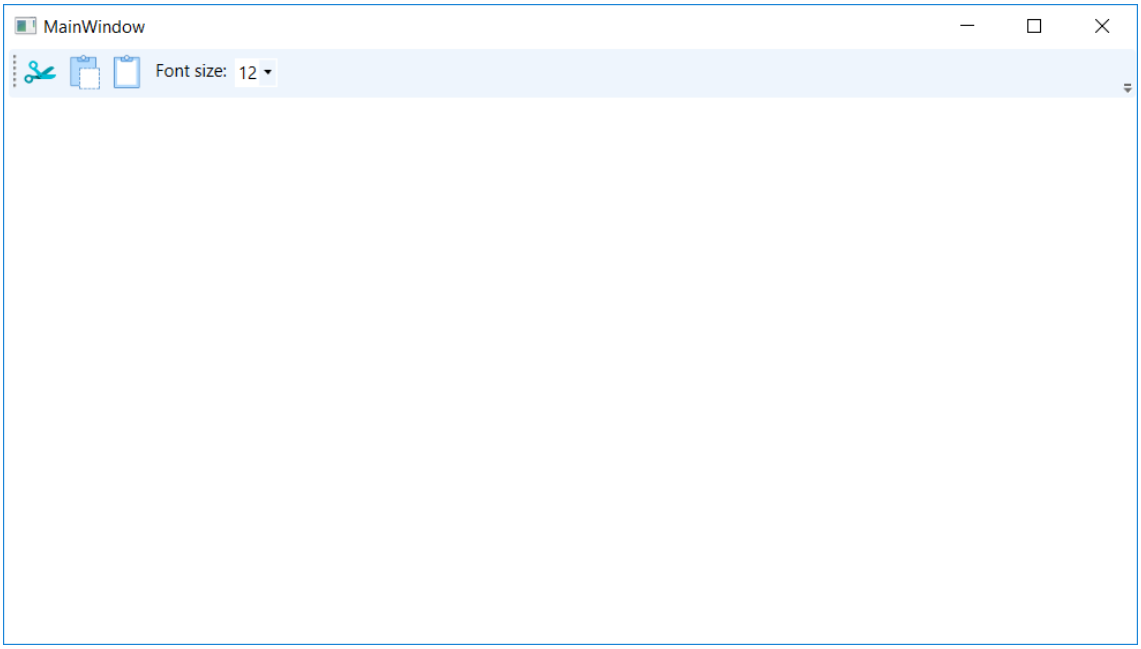


También podemos añadir componentes distintos, como por ejemplo un combobox

```
<ToolBar>
        <Label>Font size:</Label>
        <ComboBox>
              <ComboBoxItem>10</ComboBoxItem>
              <ComboBoxItem IsSelected="True">12</ComboBoxItem>
              <ComboBoxItem>14</ComboBoxItem>
              <ComboBoxItem>16</ComboBoxItem>
        </ComboBox>
</ToolBar>
```

Cada botón tendrá asociado un manejador de eventos que realizará las acciones oportunas.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



2.3. Ribbon

Es un componente de reciente incorporación puesto que surgió con el programa Office 2007. Se trata de un panel superior donde se estructuran las distintas secciones por pestañas. Dentro de cada pestaña se tienen muchas opciones, las más habituales de la sección.

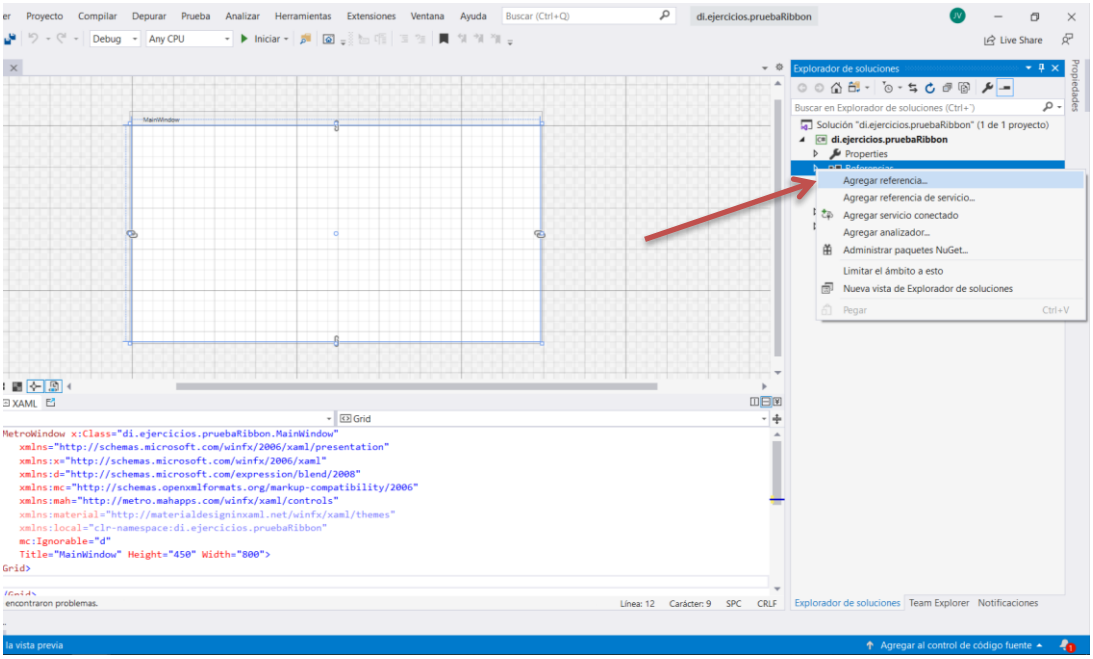
Podemos agrupar las opciones por cada sección en grupos de componentes. También tenemos la posibilidad de desplegar más opciones de las mostradas.

Este tipo de componentes persiguen mostrar de manera rápida las principales opciones utilizadas de un programa, mientras que las menos usadas permanecen ocultas salvo que queramos utilizarlas.

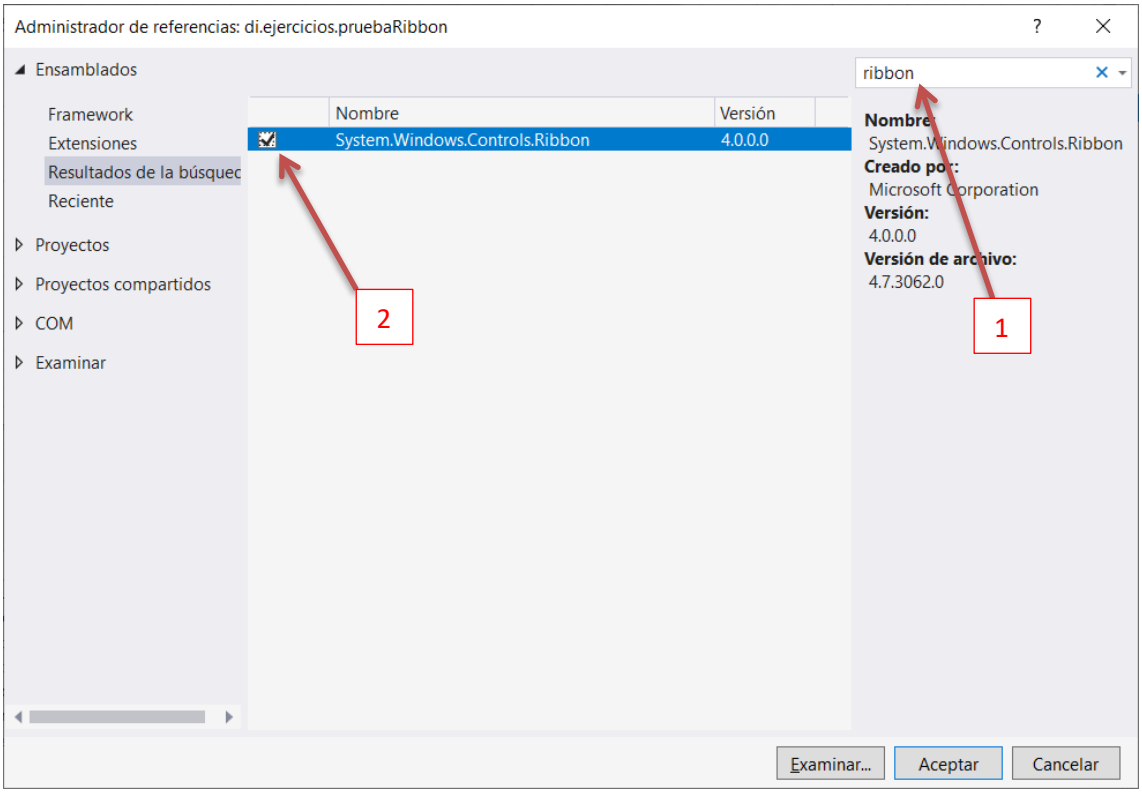
Podemos utilizar el control que proporciona Microsoft, o bien uno más moderno, llamado **Fluent Ribbon**.

2.3.1. Microsoft

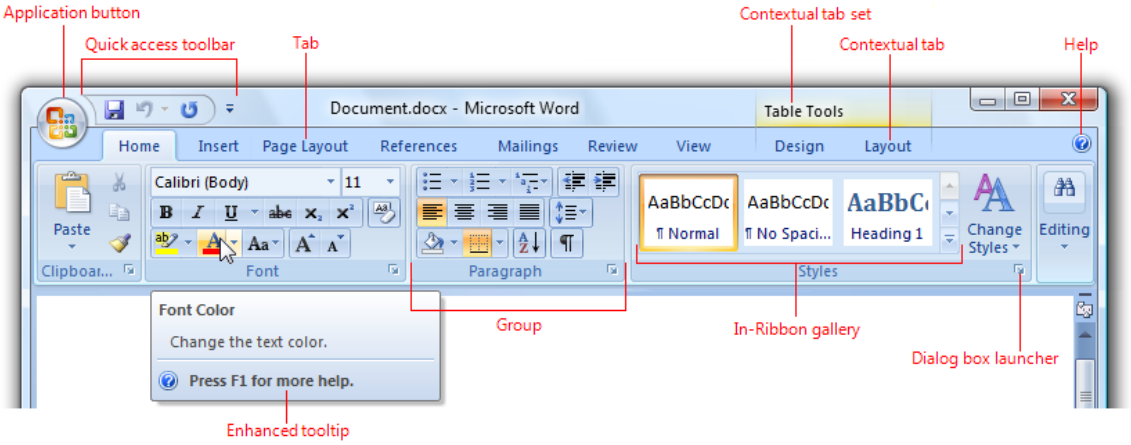
Por defecto WPF no tiene un componente Ribbon, si queremos disponer de uno debemos añadir una **Referencia** nueva a nuestro proyecto. Para ello, en el **Explorador de Soluciones**, seleccionamos ***Referencias*** y apretamos el botón derecho del ratón. Seleccionamos la primera opción.



Después buscamos el control en la parte derecha superior y seleccionamos la opción de ***System.Windows.Controls.Ribbon*** y pulsamos aceptar. Ahora ya tenemos añadida la referencia a nuestro proyecto.



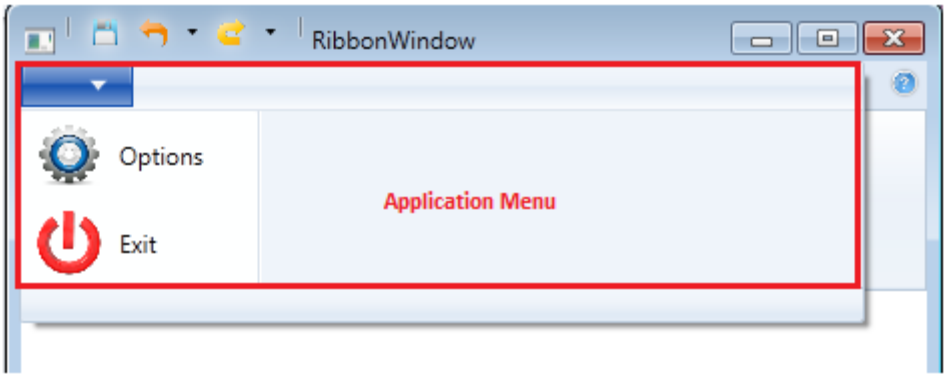
Se trata de un componente complejo con muchas opciones y controles asociados. Vamos a explicarlas brevemente:



- **HelpPaneContent**: se trata de un panel para situar botones de ayuda. Se encuentra en la parte superior derecha.

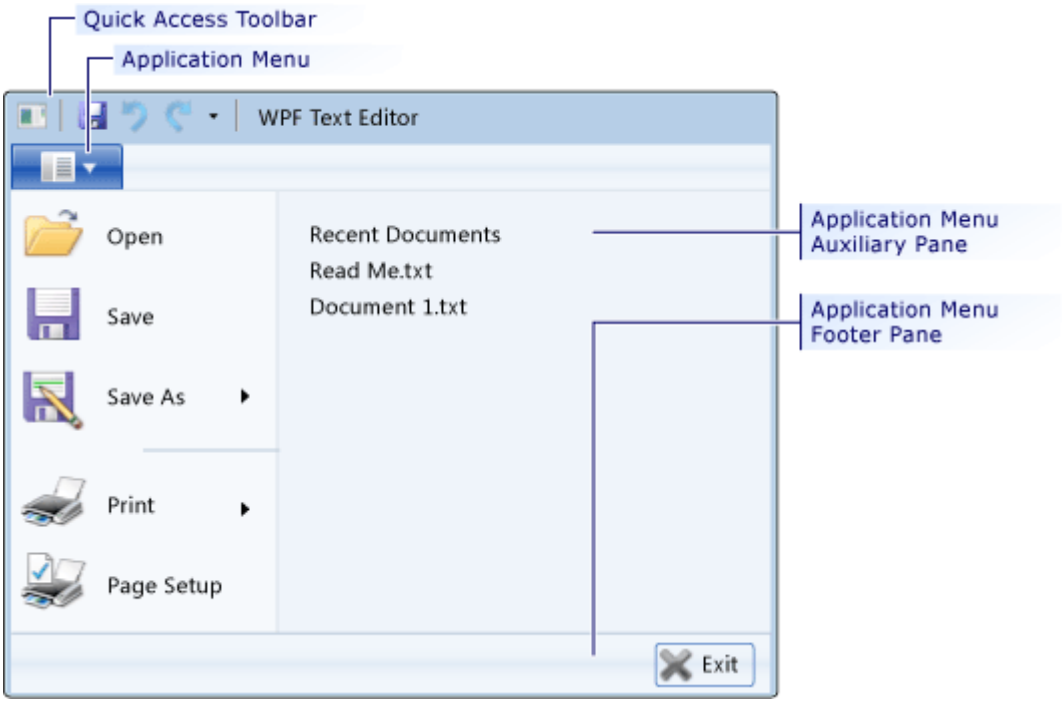
```
<Ribbon.HelpPaneContent>
    <RibbonButton SmallImageSource="/Iconos/help.png" />
</Ribbon.HelpPaneContent>
```

- **Application menú**: en la primera pestaña siempre se ubica este control que se utiliza para incluir opciones relacionadas con los ficheros como *Abrir, Guardar, Imprimir, …*



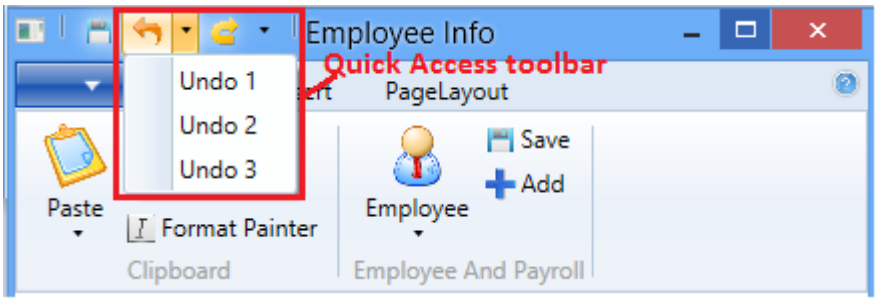

```
<Ribbon.ApplicationMenu>
   <RibbonApplicationMenu KeyTip="F"
       SmallImageSource="Iconos/menu.png">
       <RibbonApplicationMenuItem Header="Options"
              ImageSource="Iconos/options.png" />
       <RibbonApplicationMenuItem Header="Exit"
              ImageSource="Iconos/quit.png" />
   </RibbonApplicationMenu>
</Ribbon.ApplicationMenu>
```

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



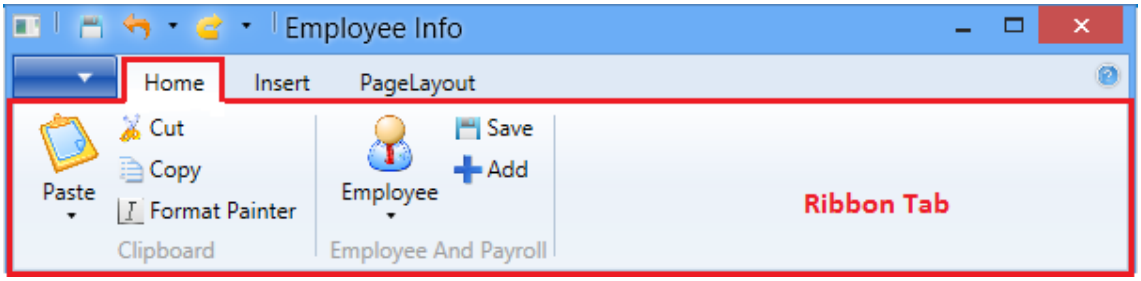
- **Quick Access Toolbar**: la barra de acceso rápido es una barra de herramientas pequeña y personalizable que proporciona acceso a **comandos de uso frecuente**. Estos comandos son independientes de la pestaña seleccionada. Por defecto, la barra de acceso rápido se encuentra en la barra de título de la ventana de la aplicación, pero puede ser configurado para visualizarla debajo de la cinta. Normalmente se incluyen opciones muy utilizadas que deben de verse independientemente de la pestaña seleccionada. En el caso de Office se añade la opción de **Guardar, Redo** y **Undo**.

```
<Ribbon.QuickAccessToolBar>
    <RibbonQuickAccessToolBar>
        <RibbonButton x:Name ="Save" LargeImageSource="Iconos/save.png" />
        <RibbonSplitButton x:Name ="Undo" SmallImageSource="Iconos/undo.png">
        <RibbonSplitMenuItem Header="Undo 1" />
        <RibbonSplitMenuItem Header="Undo 2" />
        <RibbonSplitMenuItem Header="Undo 3" />
        </RibbonSplitButton>
        <RibbonSplitButton x:Name="Redo" SmallImageSource="Iconos/redo.png" >
            <RibbonSplitMenuItem Header="Redo 1" />
            <RibbonSplitMenuItem Header="Redo 2" />
        </RibbonSplitButton>
    </RibbonQuickAccessToolBar>
</Ribbon.QuickAccessToolBar>
```



- **Tab**: se trata de una pestaña donde se ubican las funcionalidades de la aplicación. Se organizan las opciones por categorías y dentro de cada pestaña se ubican las funciones de una categoría. Es una forma similar de organización a la de los menús.

```
<!—Ribbon Tab #1: -->
<RibbonTab Header="Home" KeyTip="H" >
</RibbonTab>
<!-- Ribbon Tab #2: -->
<RibbonTab Header="Insert" KeyTip="I">
</RibbonTab>
<!-- Ribbon Tab #3: -->
<RibbonTab Header="PageLayout" KeyTip="L">
</RibbonTab>
```



- **Group**: dentro de una pestaña o **tab** podemos agrupar funciones similares, como si fuera un submenú. A estas divisiones se les denominan *Grupos*. De esta manera podemos aumentar el nivel de jerarquía y organización de nuestras opciones.



C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<!-- Home  group-->
<RibbonGroup x:Name="ClipboardGroup" Header="Home">
   <RibbonMenuButton LargeImageSource="Iconos/paste.png" Label="Paste">
      <RibbonMenuItem ImageSource="Iconos/paste.png" Header="Keep Text Only"/>
      <RibbonMenuItem ImageSource="Iconos/paste.png" Header="Paste
          Special..."/>
   </RibbonMenuButton>
   <RibbonButton SmallImageSource="Iconos/cut.png" Label="Cut" />
   <RibbonButton SmallImageSource="Iconos/copy.png" Label="Copy" />
   <RibbonButton SmallImageSource="Iconos/format_painter.png" Label="Format
       Painter"/>
</RibbonGroup>
<!-- Employee And Payroll group-->
<RibbonGroup x:Name="Employee" Header="Employee And Payroll">
   <RibbonMenuButton LargeImageSource="Iconos/personal.png" Label="Employee" >
      <RibbonMenuItem ImageSource="Iconos/paste.png" Header="Keep Text Only"/>
      <RibbonMenuItem ImageSource="Iconos/paste.png" Header="Paste
        Special..."/>
   </RibbonMenuButton>
   <RibbonButton SmallImageSource="Iconos/save.png" Label="Save" />
   <RibbonButton SmallImageSource="Iconos/add.png" Label="Add" KeyTip="C" />
</RibbonGroup>
```

- **Dialog box launcher**: es una pequeña flechita que se ubica normalmente en la parte inferior derecha de un grupo, y que se utiliza para desplegar un diálogo con más opciones asociadas. Está opción no está soportada por el componente Ribbon que viene por defecto.

Luego los componentes que se añaden al control Ribbon son específicos y están diseñados para encajar de forma adecuada en el componente. A continuación se muestra un listado de los que podemos encontrar:

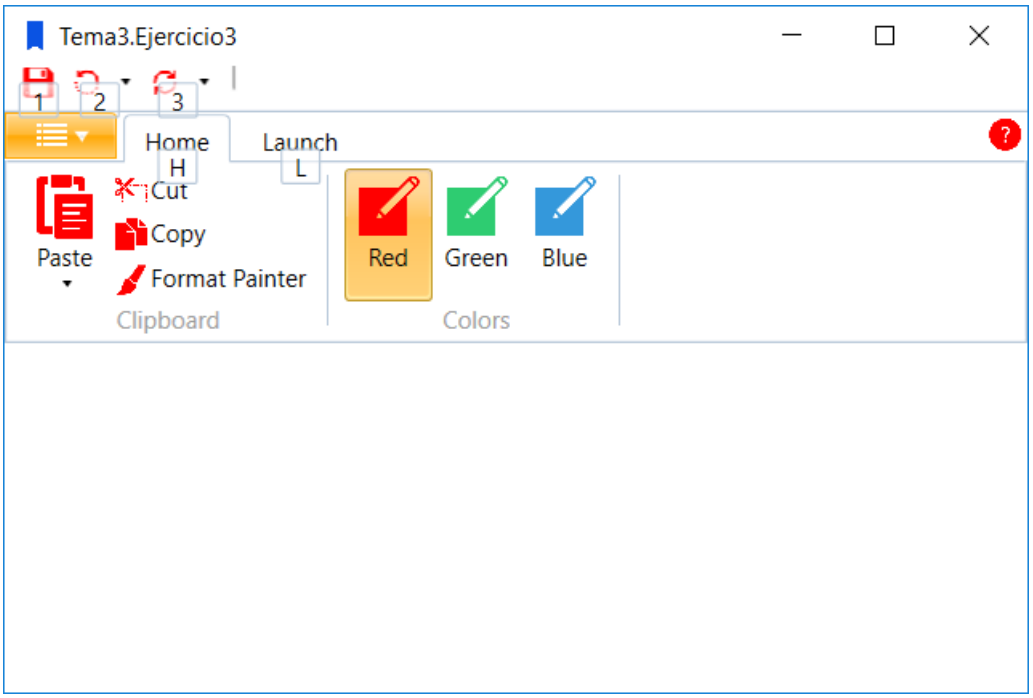
- **RibbonButton**: se trata de un botón, normalmente con un icono
- **RibbonCheckBox**: un checkbox
- **RibbonComboBox**: una lista desplegable o combobox para desplegar varias opciones
- **RibbonMenuButton**: un botón que al apretarlo despliega un menú contextual. Suele ir acompañado de una imagen grande.
- **RibbonMenuButtonItem**: ítem del menú que se despliega al apretar el RibbonMenuButton
- **RibbonRadioButton**: especifica una opción a elegir mediante un radiobutton
- **RibbonSplitButton**: es muy similar al menubutton, tiene una imagen o icono y una flecha que despliega un menú, sin embargo, mientras que en el componente **RibbonMenuItem** al apretar sobre él se dispara el evento de desplegarlo simplemente. El control **RibbonSplitMenu** dispara este mismo evento al apretar la flecha, es decir, despliega el menú, sin embargo añade un nuevo evento que se dispara al hacer click sobre la imagen.
- **RibbonSplitButtonItem**: ítem del menú que se despliega
- **RibbonTextBox**: una caja de texto para introducir información

- **RibbonToggleButton**: un conjunto de botones, de los cuales solamente uno de ellos está activo en un momento determinado.

Se trata de componentes comunes que se han adaptado al control **Ribbon**.

Por último destacar que este control también tiene la posibilidad de utilizarse solamente con el teclado. Para ello utilizaremos una propiedad de los componentes que se llama **KeyTip**.

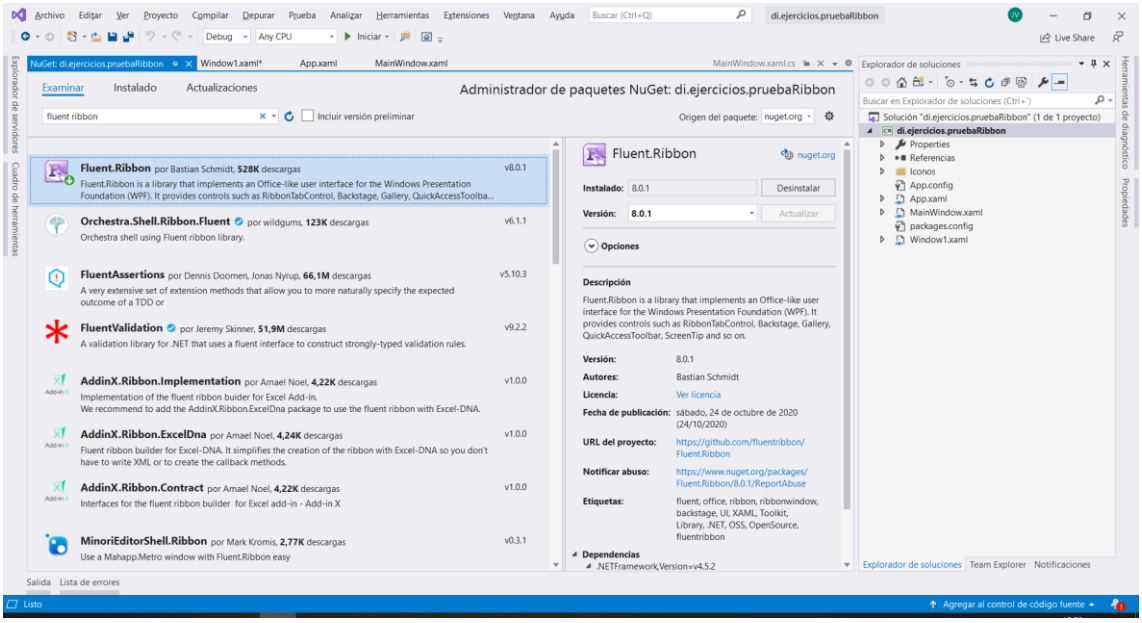
Esta propiedad implementa los *aceleradores* de teclado para acceder a los controles mediante una combinación de teclado. Aparecen en cada control al apretar la tecla **Alt**. La combinación de teclas tiene que ser como máximo de tres caracteres sin espacios, tabulaciones o saltos de línea.



Hay que tener cuidado con las combinaciones elegidas para evitar conflictos con los atajos de teclado definidos por el sistema.

2.3.2. Fluent Ribbon

En este caso utilizaremos el control proporcionado por Fluent. Es prácticamente igual al anterior, pero con un aspecto más moderno. Se trata de un control que se encuentra en el repositorio de nuget, por lo que podemos instalarlo de una manera sencilla.



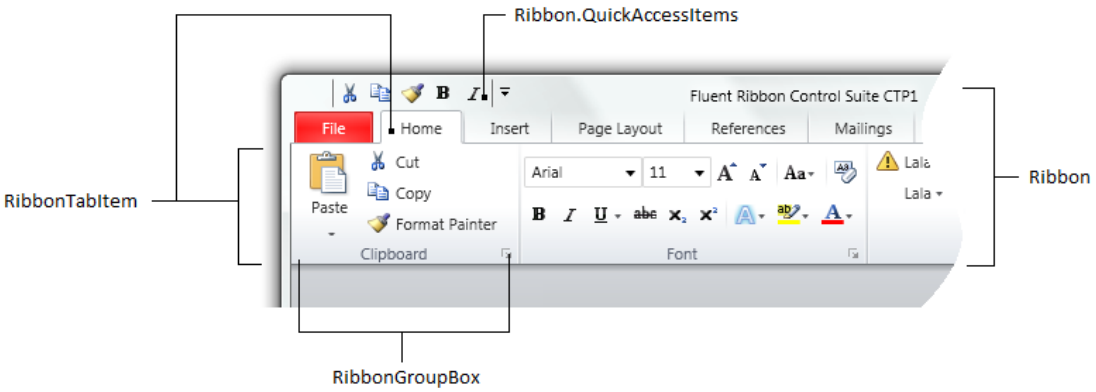
Tras instalarlo, debemos añadir los estilos en nuestro fichero de configuración App.xaml:

```
<!-- Estilos para Fluent Ribbon -->
<ResourceDictionary

    Source="pack://application:,,,/Fluent;Component/Themes/Generic.xaml" />
```

Por último, para incorporarlo a nuestra ventana definimos el conjunto de etiquetas:

```
xmlns:fluent="urn:fluent-ribbon"
```



El aspecto es más moderno, pero aunque no tiene las mismas partes que el anterior control que hemos visto, si dispone de las principales.

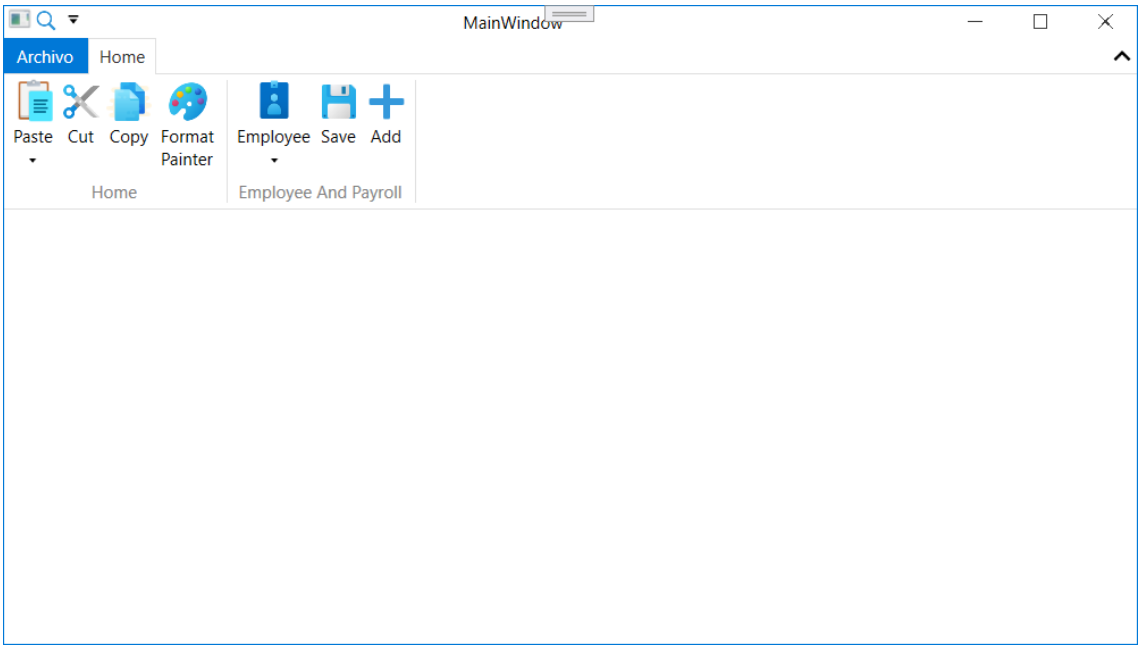
No dispone del botón de ayuda, pero sí de las opciones de acceso rápido, aunque para poder verlas debemos cambiar el tipo de ventana a **Fluent:RibbonWindow**, de lo contrario no las podremos ver.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

La parte que se refiere a Application Menu se llama **Menu**. A continuación ponemos un ejemplo:

```
<Fluent:RibbonWindow x:Class="di.ejercicios.prueba.fluent.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:Fluent="urn:fluent-ribbon"
        xmlns:local="clr-namespace:di.ejercicios.prueba.fluent"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
    <Grid>
        <Fluent:Ribbon>
            <Fluent:Ribbon.QuickAccessItems>
                <!--Use Content or Target Property to set QAT item-->
                <Fluent:QuickAccessMenuItem IsChecked="true">
                    <Fluent:Button Header="Pink" Icon="Iconos/buscar.png" />
                </Fluent:QuickAccessMenuItem>
            </Fluent:Ribbon.QuickAccessItems>
            <Fluent:Ribbon.Menu>
                <Fluent:Backstage>
                    <Fluent:BackstageTabControl>
                        <Fluent:BackstageTabItem Header="Opciones" />
                        <Fluent:Button Header="Cerrar"
                                LargeIcon="Iconos/quit.png"  />
                    </Fluent:BackstageTabControl>
                </Fluent:Backstage>
            </Fluent:Ribbon.Menu>
            <Fluent:RibbonTabItem Header="Home"  >
                <!-- Home  group-->
                <Fluent:RibbonGroupBox x:Name="ClipboardGroup" Header="Home">
                    <Fluent:SplitButton LargeIcon="Iconos/paste.png"
                                Header="Paste" >
                        <Fluent:MenuItem Icon="Iconos/paste.png" Header="Keep
                                Text Only" />
                        <Fluent:MenuItem Icon="Iconos/paste.png" Header="Paste
                                Special..." KeyTip="S"/>
                    </Fluent:SplitButton>
                    <Fluent:Button LargeIcon="Iconos/cut.png" Header="Cut"/>
                    <Fluent:Button LargeIcon="Iconos/copy.png" Header="Copy"/>
                    <Fluent:Button LargeIcon="Iconos/format_painter.png"
                                Header="Format Painter"/>
                </Fluent:RibbonGroupBox>
                <!-- Employee And Payroll group-->
                <Fluent:RibbonGroupBox x:Name="Employee" Header="Employee And
                        Payroll">
                    <Fluent:SplitButton LargeIcon="Iconos/personal.png"
                                Header="Employee">
                        <Fluent:MenuItem Icon="Iconos/paste.png" Header="Keep
                                Text Only" />
                        <Fluent:MenuItem Icon="Iconos/paste.png" Header="Paste
                                Special..." />
                    </Fluent:SplitButton>
                    <Fluent:Button LargeIcon="Iconos/save.png" Header="Save"/>
                    <Fluent:Button LargeIcon="Iconos/add.png" Header="Add" />
                </Fluent:RibbonGroupBox>
            </Fluent:RibbonTabItem>
        </Fluent:Ribbon>
    </Grid>
</Fluent:RibbonWindow>
```

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



2.4. Tipo desplegable o acordeón

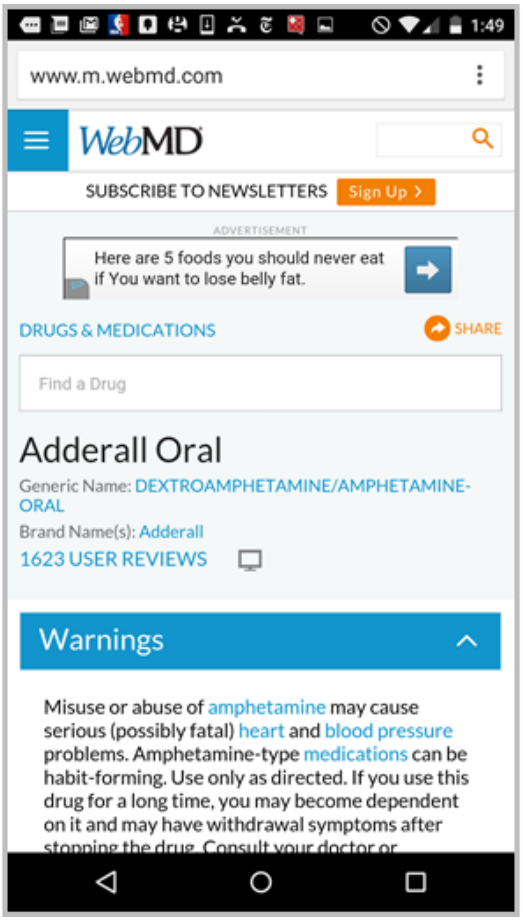
Este elemento se suele disponer en la parte izquierda y consta de una serie de paneles que se pueden contraer o desplegar en función de las opciones que queramos utilizar.

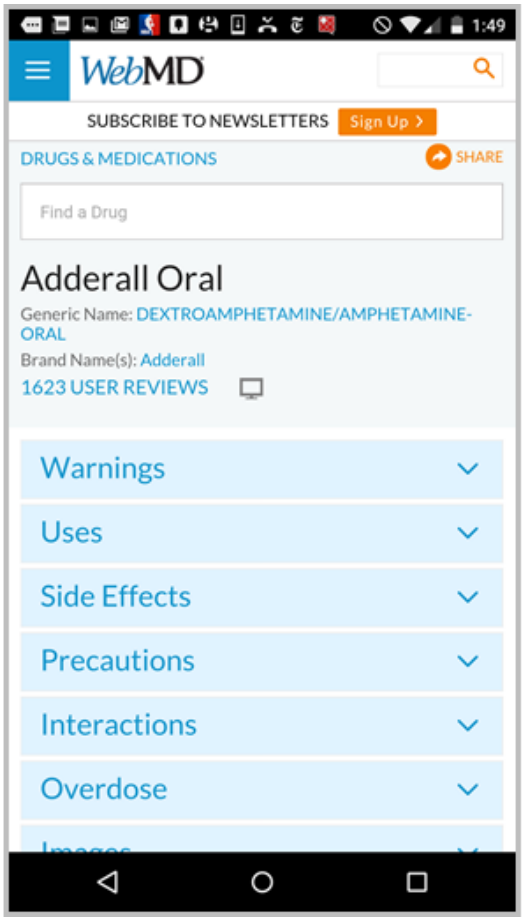
En cada panel podemos colocar las opciones que tienen características o funcionalidades comunes.

Este tipo de controles también pueden ubicarse a la derecha o como complemento de un menú en caso de programas con multitud de opciones.

Este tipo de componentes suelen utilizarse en las webs actuales, sobre todo si están capacitadas para adecuarse a distintos tipos de dispositivos de acceso a la web (***Responsive***) como tablets móviles u ordenadores.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es





WPF no lleva este control de serie, por lo tanto debemos utilizar un componente de terceros para implementar está función. Existen varias empresas que tienen controles de este tipo, pero suelen ser de pago. Nosotros utilizaremos un conjunto de componentes gratuitos que se encuentran en ***Extended WPF Toolkit***.

En el anexo se encuentra como activar este control.

Este control tiene principalmente dos clases que son las que tendremos que utilizar para representarlo en pantalla.

La primera es **Accordion** que es la principal y la que define el componente. Suele resultar interesante definir la propiedad *VerticalAligment* como *Stretch* para que ocupe todo el espacio vertical.

En este control cabe destacar la propiedad *ExpandDirection* que marca la dirección en la que se despliega el acordeón. Puede ser hacia arriba, abajo, derecha o izquierda.

Este componente simplemente es el contenedor de los paneles desplegables que tienen como nombre **AccordionItem**. Esta clase dibuja un panel junto con un botón que esconde o despliega el panel al apretarlo.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<layoutToolkit:Accordion>
    <layoutToolkit:AccordionItem Header="Red">
        <Rectangle Fill="Red" Height="120" Width="200" />
    </layoutToolkit:AccordionItem>
    <layoutToolkit:AccordionItem Header="Orange">
        <Rectangle Fill="Orange" Height="120" Width="200" />
    </layoutToolkit:AccordionItem>
    <layoutToolkit:AccordionItem Header="Yellow">
        <Rectangle Fill="Yellow" Height="120" Width="200" />
    </layoutToolkit:AccordionItem>
    <layoutToolkit:AccordionItem Header="Green">
        <Rectangle Fill="Green" Height="120" Width="200" />
    </layoutToolkit:AccordionItem>
    <layoutToolkit:AccordionItem Header="Blue">
        <Rectangle Fill="Blue" Height="120" Width="200" />
    </layoutToolkit:AccordionItem>
    <layoutToolkit:AccordionItem Header="Indigo">
        <Rectangle Fill="Indigo" Height="120" Width="200" />
    </layoutToolkit:AccordionItem>
    <layoutToolkit:AccordionItem Header="Violet">
        <Rectangle Fill="Violet" Height="120" Width="200" />
    </layoutToolkit:AccordionItem>
</layoutToolkit:Accordion>
```

En el ejemplo anterior mostrará un acordeón que tendrá rectángulos de colores como contenido.

Dentro del componente **AccordionItem** podemos añadir layouts y dentro de éstos añadir otros componentes.

Este control está discontinuado y no ofrece actualizaciones, por ello tiene un aspecto algo antiguo, a la vez que resulta complicado configurarlo bien.

A continuación veremos un control similar con el control **Expander** y los estilos de Material Design.

2.4.1. Expander

Este control tiene una flecha que indica la dirección en la que el control, como su nombre indica, se expande. En este caso nos interesa que sea hacia abajo.

Además utilizaremos el control ListView que muestra una lista de elementos de manera seguida, similar a un StackPanel.

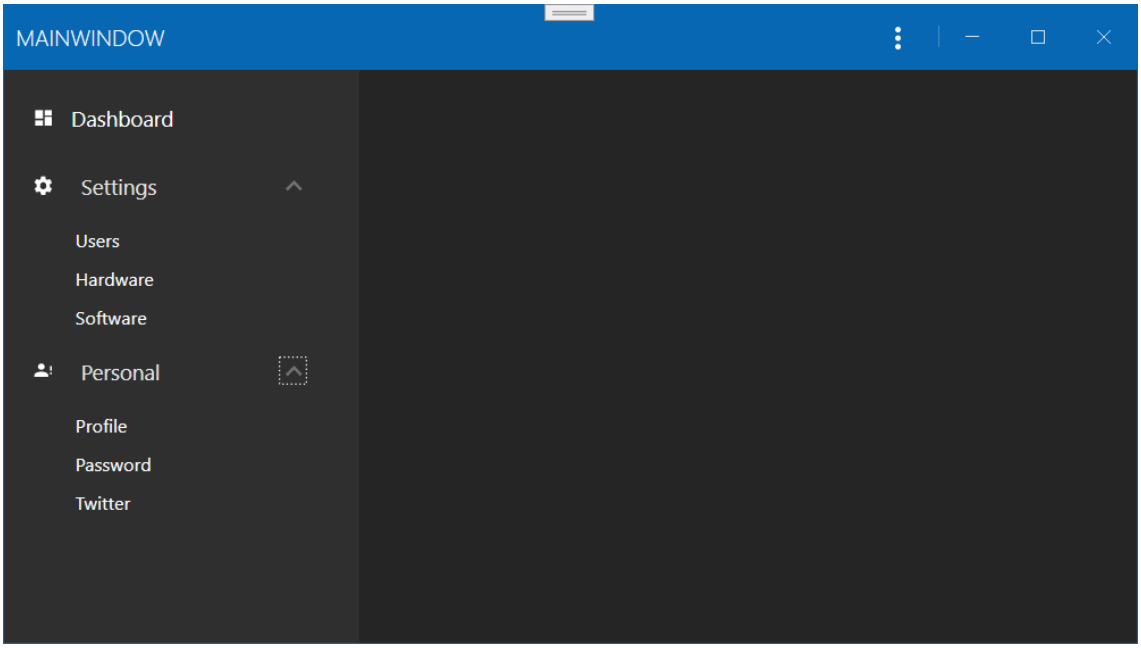
A continuación mostramos un ejemplo:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<DockPanel>
   <Grid DockPanel.Dock="Left" Background="#E5303030" Width="250">
       <StackPanel x:Name="Menu" Margin="10">
          <Grid>
          <!-- Definimos el titulo del panel -->
            <material:PackIcon Kind="ViewDashboard" Width="15" Height="15"
Margin="10 16" Foreground="White"/>
<!-- Ahora utilizamos este control que ns muestra un elemento de la lista -
->
            <ListBoxItem x:Name="ListViewItemMenu" Content="Dashboard"
Padding="37 14" FontSize="15" Foreground="White"
Style="{StaticResource MaterialDesignListBoxItem}"/>
           </Grid>
           <Grid>
<!-- Ahora mostramos el icono de la sección -->
              <material:PackIcon Kind="Settings" Width="15" Height="15"
Margin="10 16" Foreground="White" />
<!-- Utilizamos elcontrol Expander para la primera sección -->
              <Expander IsExpanded="False" Width="210"
HorizontalAlignment="Right" Style="{StaticResource MaterialDesignExpander}"
Header="Settings" Background="{x:Null}" Foreground="White" >
<!-- Utilizamos este control para mostrar la lista de opciones de la
sección -->
                <ListView x:Name="ListViewMenu" ItemsSource="{Binding
Path=SubItems}" Foreground="White" Style="{StaticResource
MaterialDesignListView}"
ScrollViewer.HorizontalScrollBarVisibility="Disabled">
                  <ListViewItem Content="Users" Padding="20 5" />
                  <ListViewItem Content="Hardware" Padding="20 5" />
                  <ListViewItem Content="Software" Padding="20 5" />
                 </ListView>
               </Expander>
           </Grid>
           <Grid>
<!-- Repetimos lo mismo para la siguiente sección -->
             <material:PackIcon Kind="UserAlert" Width="15" Height="15"
Margin="10 16" Foreground="White" />
               <Expander IsExpanded="False" Width="210"
HorizontalAlignment="Right" Header="Personal"
Style="{StaticResource MaterialDesignExpander}"
Background="{x:Null}" Foreground="White">
                  <ListView ItemsSource="{Binding Path=SubItems}"
                   Foreground="White" Style="{StaticResource
MaterialDesignListView}"
ScrollViewer.HorizontalScrollBarVisibility="Disabled">
                    <ListViewItem Content="Profile" Padding="20 5" />
                    <ListViewItem Content="Password" Padding="20 5" />
                    <ListViewItem Content="Twitter" Padding="20 5" />
                  </ListView>
                </Expander>
            </Grid>
       </StackPanel>
   </Grid >
   <Grid  x:Name="gridCentral">
   </Grid>
</DockPanel>
```

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

El resultado es el siguiente:



2.5. Pestañas

Las pestañas nos permiten separar distintos contenidos dentro de una misma funcionalidad. Normalmente se utilizan como apoyo para poder diferenciar la información mostrada en compartimentos clasificados según la funcionalidad o el tipo de información que muestran.

No resulta eficaz realizar una interfaz únicamente con pestañas, como ya he dicho se utiliza como apoyo de las otras formas de organización de la aplicación. Realizar una aplicación solamente con pestañas implica que la aplicación resulte poco escalable.

El componente para WPF es **TabControl** que es el contenedor de las pestañas. Este control nos permite disponer las pestañas de cuatro formas distintas:

- En la parte superior en horizontal. Esta es la **opción por defecto**.
- En la parte inferior en horizontal
- En la parte izquierda en vertical
- En la parte derecha en vertical

Para cambiar el aspecto por defecto dela orientación de las pestañas utilizamos la propiedad **TabStripPlacement**.

Como ya hemos dicho este componente es un contenedor de controles **TabItem** que representan una pestaña.

En un componente TabControl colocaremos tantos TabItems como pestañas tengamos. La propiedad **Header** nos proporciona el título de la pestaña. Incluso podemos colocar paneles dentro de la cabecera para añadir texto e imágenes.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Podemos recorrer las pestañas que tiene el contenedor **TabControl** con la propiedad *Items*.

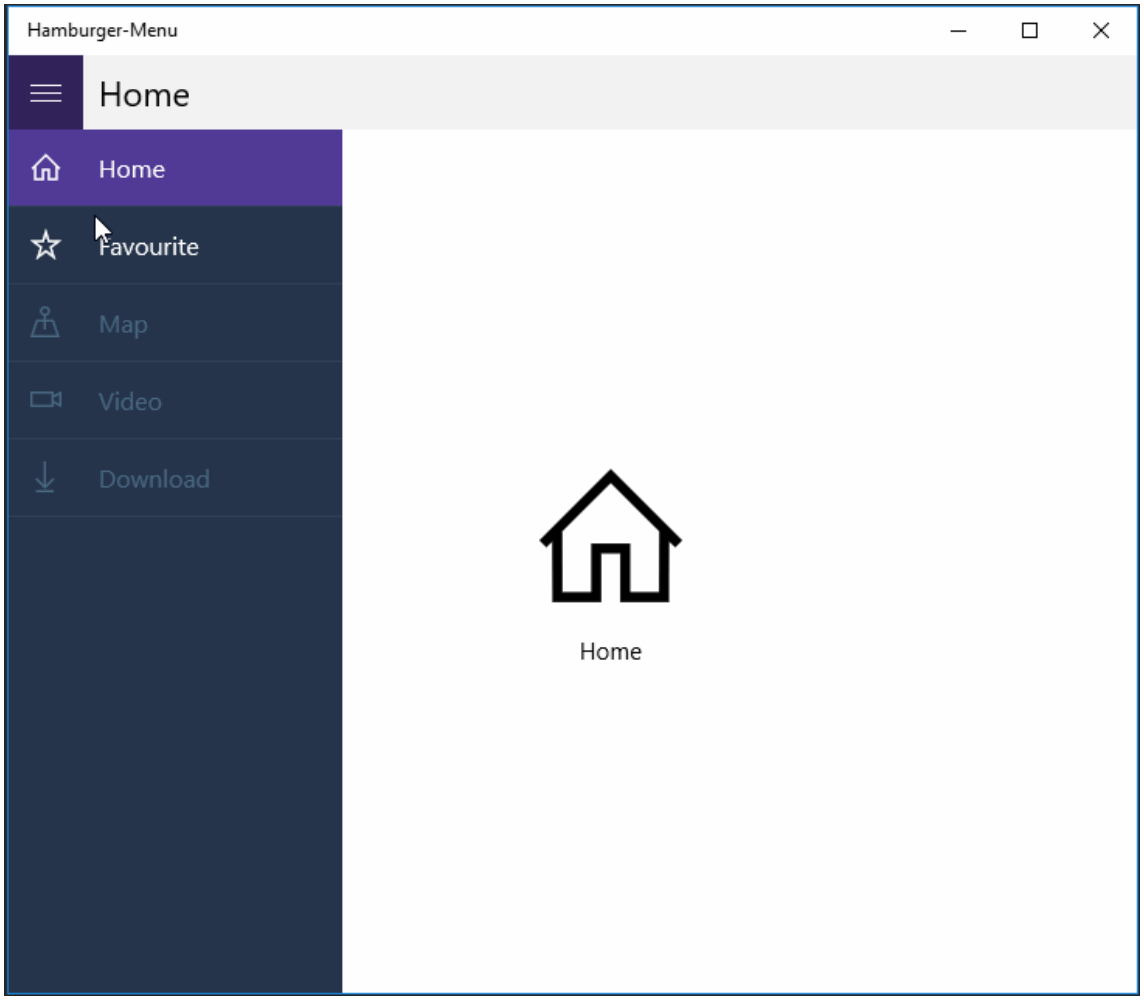
Este control tiene múltiples eventos asociados pero el que más suele utilizarse es el que se dispara al cambiar de pestaña. Este evento es **SelectionChanged**.

2.6. Hamburguer menú

Se trata de un menú ocultable que se muestra al apretar el botón de las tres rayas, por este icono se llama así a este componente.

Se encuentra normalmente en la esquina izquierda superior y se expande de izquierda a derecha. Se compone de una serie de opciones principales que al apretar nos muestran un panel específico en la parte central.

Por defecto, en WPF, no se dispone de este control y se debe de añadir mediante la librería auxiliar de componentes y estilos **MahApps**. En el anexo II se explica cómo instalarlo y configurarlo.



C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Vamos con un ejemplo. Primero debemos añadir la librería de etiquetas en nuestra ventana principal:

```
xmlns:mah="http://metro.mahapps.com/winfx/xaml/controls"
```

Después utilizaremos el componente y nos aparece por defecto en nuestro diseñador de interfaces.

Ahora vamos a añadir los elementos que aparecerán en nuestro menú desplegable. Pero para ello utilizaremos en primer lugar un *DataTemplate* para maquetar los elementos del menú. Al igual que sucede en las páginas web con CSS podemos tener un fichero que modifique la forma de ver los componentes, para ello utilizaremos los Template o platillas de WPF, que nos permiten definir la forma en la que se verán nuestros controles.

En este caso utilizamos unas plantillas para ver los elementos del HamburguerMenu. Para ello utilizaremos un fichero de estilos propio para implementar este tipo de interfaz. En el fichero encontramos tres tipos de estilos:

- **HamburguerMenuGlyphItem**: en este caso utilizamos la fuente Segoe para mostrar iconos.
- **HamburguerMenuIconItem**: en esta plantilla utilizamos *PackIcon* para mostrar los iconos.
- **HamburguerMenuImageItem**: aquí utilizaremos rutas para obtener los iconos en formato png.

En todas estas plantillas utilizamos un Grid con dos columnas `para que el contenido se muestre de manera homogénea. Veamos un ejemplo:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<DataTemplate x:Key="MenuItemTemplate" DataType="{x:Type
mah:HamburgerMenuGlyphItem}">
<!-- Para mostrar correctamente el contenido utilizamos un Grid con dos
columnas -->
   <Grid Height="48">
      <Grid.ColumnDefinitions>
         <ColumnDefinition Width="48" />
         <ColumnDefinition />
      </Grid.ColumnDefinitions>
<!-- En este textblock se muestra el icono. Se utiliza la fuente Segoe -->
      <TextBlock Grid.Column="0"
         FontSize="16"
         HorizontalAlignment="Center"
         VerticalAlignment="Center"
         FontFamily="Segoe MDL2 Assets"
         Foreground="White"
<!-- Aquí enlazamos el texto del control TextBlock con la propiedad Glyph
de la opción de Menú. Nos mostrará el icono -->
         Text="{Binding Glyph}" />
<!-- En este segundo textblock se muestra la etiqueta que vemos al
desplegar el menu -->
      <TextBlock Grid.Column="1"
         VerticalAlignment="Center"
         FontSize="16"
         Foreground="White"
<!-- Aquí enlazamos el texto del control TextBlock con la propiedad Label
de la opción de Menú -->
         Text="{Binding Label}" />
   </Grid>
</DataTemplate>
```

En el fichero de estilos que hay en Aules se muestra el resto de las plantillas que son similares, aunque cambia la forma de especificar la imagen o icono de la opción del menú.

Para poder utilizar estas plantillas debemos añadirlas en nuestro fichero App.XAML, tal y como hacemos con los estilos de Mahapps o Material Design.

```
<!-- Estilos propios -->
<ResourceDictionary Source="/Estilos/Estilos.xaml" />
```

Una vez hemos incorporado el control y los estilos podemos empezar a utilizarlo. Este control tiene dos secciones diferenciadas:

- **ItemsTemplate**: son las opciones principales del programa
- **OptionsItemsTemplate**: son opciones auxiliares, como por ejemplo la ayuda de la aplicación.

En la definición del control utilizamos las plantillas que necesitemos para cada una de estas secciones.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<!-- Definimos las propiedades del control y las plantillas de las secciones -
->
<mah:HamburgerMenu Foreground="White" x:Name="hamMenuPrincipal"
   DisplayMode="CompactInline"
   ItemTemplate="{StaticResource MenuItemTemplateImage}"
   OptionsItemTemplate="{StaticResource MenuItemTemplateIcon}"
   Grid.ColumnSpan="2" >
<!-- En la sección de ItemsSource definimos las principales opciones -->
   <mah:HamburgerMenu.ItemsSource>
      <mah:HamburgerMenuItemCollection>
         <mah:HamburgerMenuImageItem Label="Personal" ToolTip="Personal">
             <mah:HamburgerMenuImageItem.Thumbnail>
                <BitmapImage UriSource="Iconos/personal.png" />
             </mah:HamburgerMenuImageItem.Thumbnail>
         </mah:HamburgerMenuImageItem>
         <mah:HamburgerMenuImageItem Label="Busqueda" ToolTip="Busqueda">
             <mah:HamburgerMenuImageItem.Thumbnail>
                <BitmapImage UriSource="Iconos/busqueda.png" />
             </mah:HamburgerMenuImageItem.Thumbnail>
         </mah:HamburgerMenuImageItem>
      </mah:HamburgerMenuItemCollection>
   </mah:HamburgerMenu.ItemsSource>
   <!--  Sección de opciones  -->
   <mah:HamburgerMenu.OptionsItemsSource>
      <mah:HamburgerMenuItemCollection>
         <mah:HamburgerMenuIconItem Icon="{material:PackIcon Kind=Help}"
Label="Ayuda">
         </mah:HamburgerMenuIconItem>
      </mah:HamburgerMenuItemCollection>
   </mah:HamburgerMenu.OptionsItemsSource>
</mah:HamburgerMenu>
```

3. Templates

Para cambiar el aspecto de los componentes tenemos los estilos y también disponemos de los *Templates*.

Aunque al final tienen el mismo fin tienen sus diferencias que se detallan en la tabla siguiente:

Estilos	Templates
Se utilizan para modificar los valores por defecto de la propiedad de un componente. Solamente podemos utilizar propiedades existentes en el control.	Permite modificar la estructura de un control al cual la plantilla es aplicada. Por ejemplo podemos modificar la forma de un botón y hacerlo redondo. También podemos modificar las propiedades de un control pero que no podrán ser modificadas por lo valores del control, solamente se podrá hacer en caso de realizar ***template-binding*** con las propiedades del control. Las principales partes de una plantilla

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

son:

- **Content Presenter**: se utiliza para cambiar la forma de visualizar el contenido del componente.
- **Header**: se utiliza para modificar la presentación de las cabeceras de aquellos componentes que las tienen como por ejemplo un **Tab Control**.
- **ItemsHost**: modifica la visualización de los objetos hijo de un componente
- **ItemContainerTemplate**: aplica una plantilla al contenedor de ítems.

Podemos modificar el comportamiento de los nuevos elementos mediante *Triggers*.

Es importante destacar que las propiedades que queremos que puedan ser modificadas por el usuario **deben de estar especificadas en la plantilla como *Template-Binding*** en caso contrario la propiedad no podrá cambiarse respecto a la plantilla original.

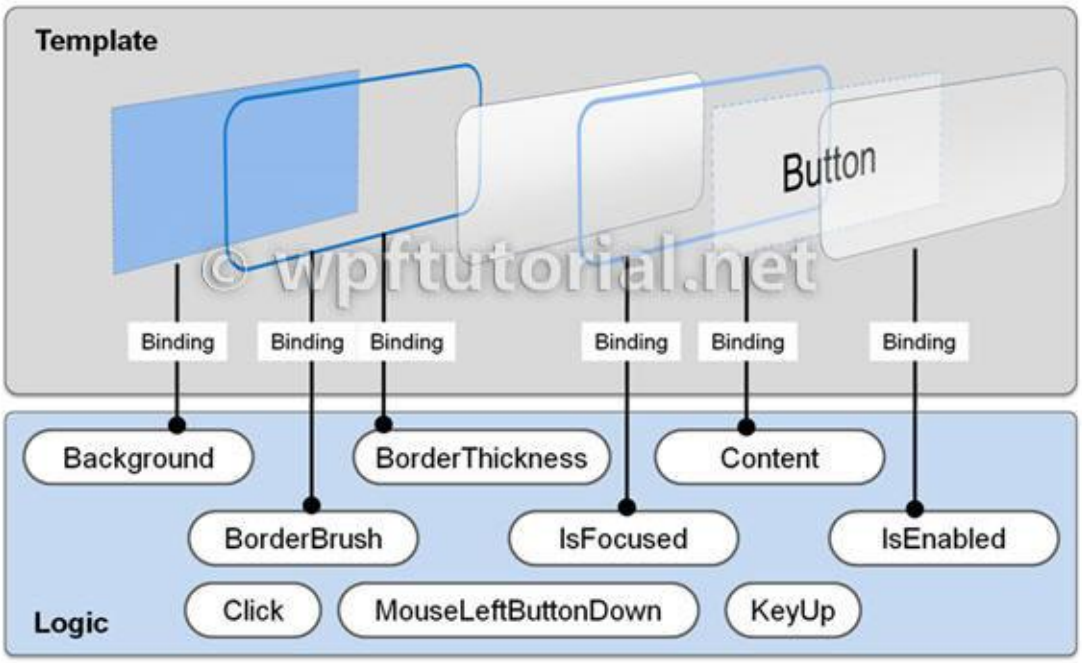
3.1. Control Template

Los controles separan la lógica (estados, eventos y propiedades) de la apariencia visual del control (template).

Los controles tienen una plantilla por defecto que determina el aspecto visual del mismo. La plantilla por defecto se adjunta junto con el control.

La plantilla es definida por una propiedad llamada **Template**. Cambiando el valor de esta propiedad por una plantilla nueva podemos cambiar el aspecto de un botón.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



Las plantillas se definen dentro de la sección **Style** del control. A continuación se muestra un ejemplo en el que se cambia la forma por defecto de un botón para que sea elíptica.

```
<Style x:Key="DialogButtonStyle" TargetType="Button">
  <Setter Property="Template">
    <Setter.Value>
      <ControlTemplate TargetType="{x:Type Button}">
        <Grid>
          <Ellipse Fill="{TemplateBinding Background}"
            Stroke="{TemplateBinding BorderBrush}"/>
              <ContentPresenter HorizontalAlignment="Center"
              VerticalAlignment="Center"/>
        </Grid>
      </ControlTemplate>
    </Setter.Value>
  </Setter>
</Style>

<Button Style="{StaticResource DialogButtonStyle}" />
```

El resultado es el siguiente:



Otro ejemplo sería el cambiar el fondo de un botón al pasar el ratón por encima:

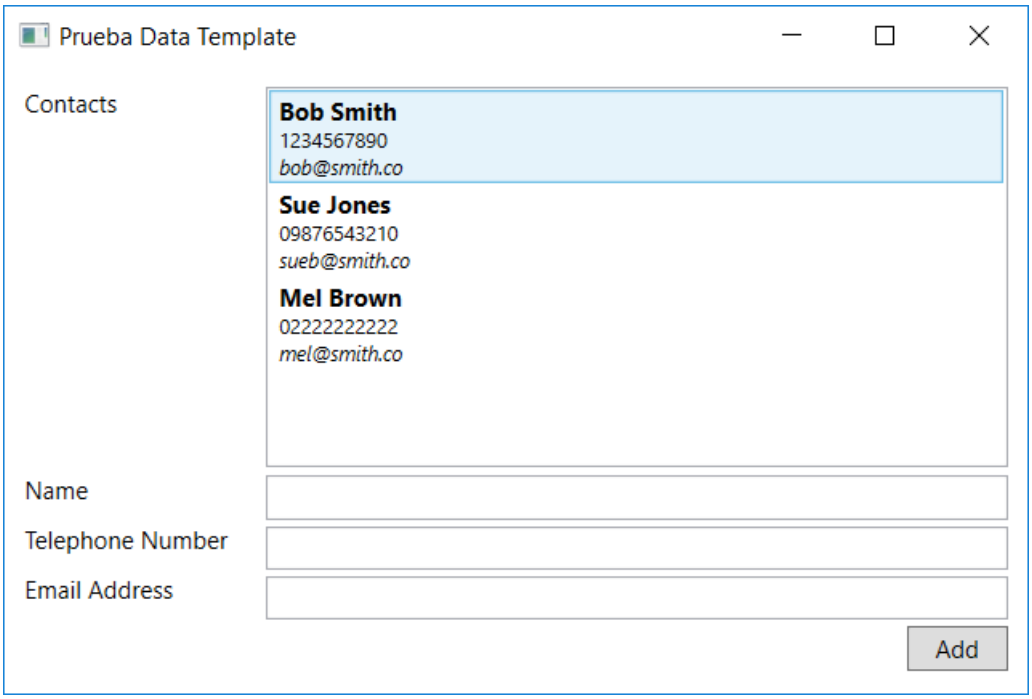
C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<Style TargetType="{x:Type Button}" x:Key="sbtnNormal">
    <Setter Property="Background" Value="LightGray"/>
    <Setter Property="Template">
      <Setter.Value>
        <ControlTemplate TargetType="{x:Type Button}">
          <Border Background="{TemplateBinding Background}">
            <ContentPresenter HorizontalAlignment="Center”
     VerticalAlignment="Center"/>
          </Border>
        </ControlTemplate>
      </Setter.Value>
      </Setter>
      <Style.Triggers>
        <Trigger Property="IsMouseOver" Value="True">
          <Setter Property="Background" Value="DarkGray"/>
        </Trigger>
      </Style.Triggers>
</Style>
```

3.2. Data Template

Se trata de un concepto similar al anterior, sin embargo está más orientado a la forma de visualizar los ítems de datos de controles tipo **ListBox, ComboBox o ListView**.

Veamos a continuación un ejemplo para ver una mejor presentación de los elementos de una lista.



Queremos mostrar los datos de nuestros contactos de la anterior forma, para ello deberemos crear una platilla de datos como la siguiente:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<ListBox.ItemTemplate>
  <DataTemplate>
    <StackPanel>
      <TextBlock FontWeight="Bold" FontSize="12" Text="{Binding
Name}"/>
      <TextBlock FontSize="10" Text="{Binding Telephone}"/>
      <TextBlock FontStyle="Italic" FontSize="10" Text="{Binding
Email}"/>
    </StackPanel>
  </DataTemplate>
</ListBox.ItemTemplate>
```

También podemos introducir **Triggers** dentro de las plantillas de datos.

```
<ListBox.ItemTemplate>
  <DataTemplate>
    <Border x:Name="bdr"
        BorderBrush="Blue" BorderThickness="1">
      <StackPanel Margin="5">
        <TextBlock x:Name="txt" FontWeight="Bold" FontSize="12"
Text="{Binding Name}"/>
        <TextBlock FontSize="10" Text="{Binding Telephone}"/>
        <TextBlock FontStyle="Italic" FontSize="10" Text="{Binding
Email}"/>
      </StackPanel>
    </Border>
    <DataTemplate.Triggers>
      <Trigger SourceName="bdr" Property="IsMouseOver" Value="True">
        <Setter TargetName="bdr" Property="Background"
Value="LightGray"/>
        <Setter TargetName="txt" Property="Foreground" Value="Red"/>
      </Trigger>
    </DataTemplate.Triggers>
  </DataTemplate>
</ListBox.ItemTemplate>
```

Al pasar el ratón por encima el fondo se vuelve gris y el nombre se pone de color rojo.

ANEXO I. INSTALACIÓN CONTROLES

WPF viene con unos controles estándar que resultan suficientes para realizar una aplicación de escritorio, sin embargo, en muchas ocasiones nos encontramos con la limitación de alguno de estos controles o simplemente que no se encuentran en el conjunto estándar.

Se pueden dar distintas situaciones, por ejemplo, un control como ***Ribbon*** no se incluye entre los comunes pero se puede activar porque Microsoft lo desarrolló después.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

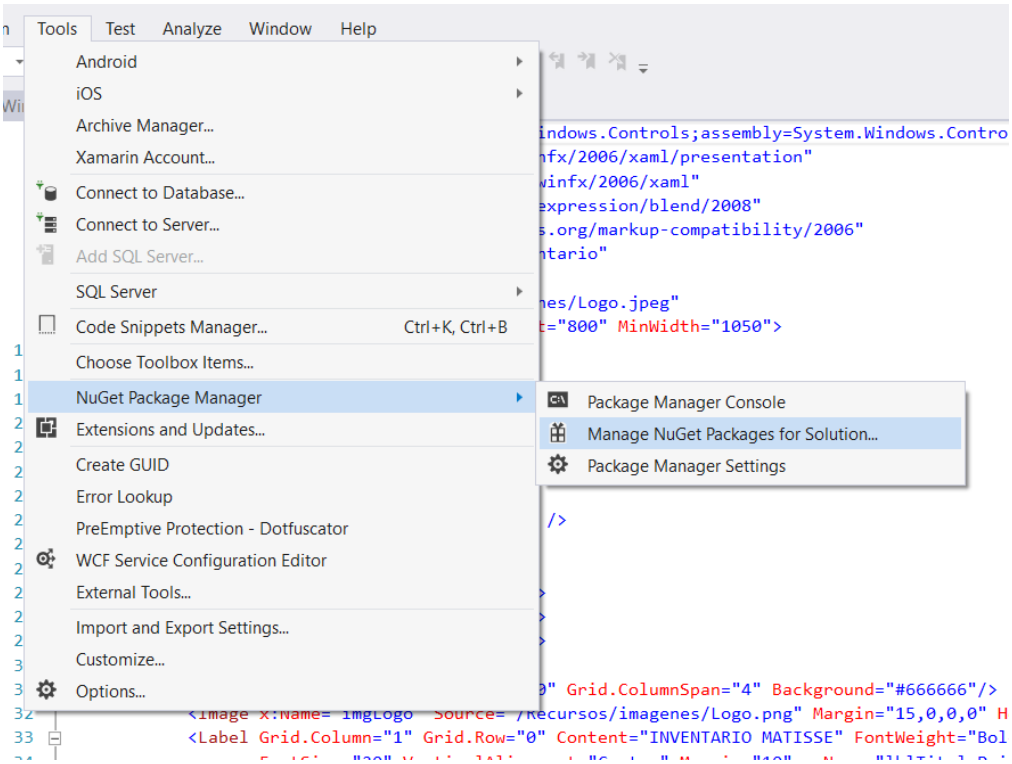
En otras ocasiones existen proyectos de software libre que aumentan el número de controles de forma gratuita como **WPFToolkit**. O bien mejoran las características de algunos ya existentes.

Y por último tenemos empresas que disponen de controles más desarrollados y sofisticados que nos permiten agilizar nuestros desarrollos. Normalmente son de pago y tienen versiones de prueba durante un tiempo.

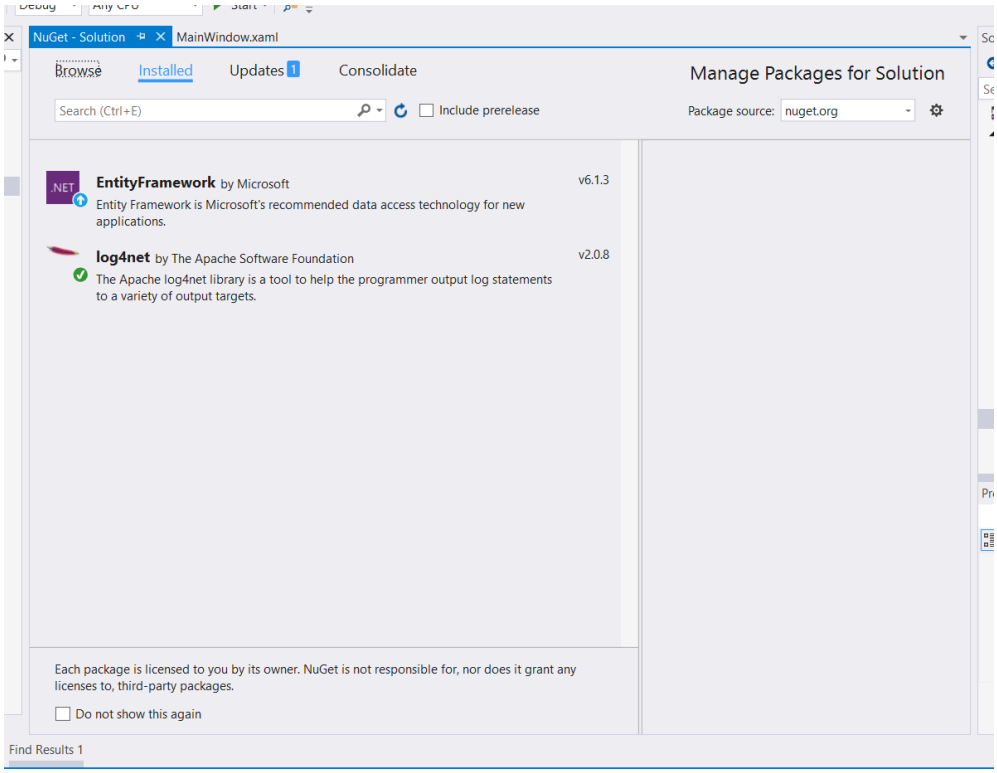
Tenemos dos opciones para instalar nuevos controles:

a) La primera arrastrando la librería que contiene los elementos a la barra de herramientas.

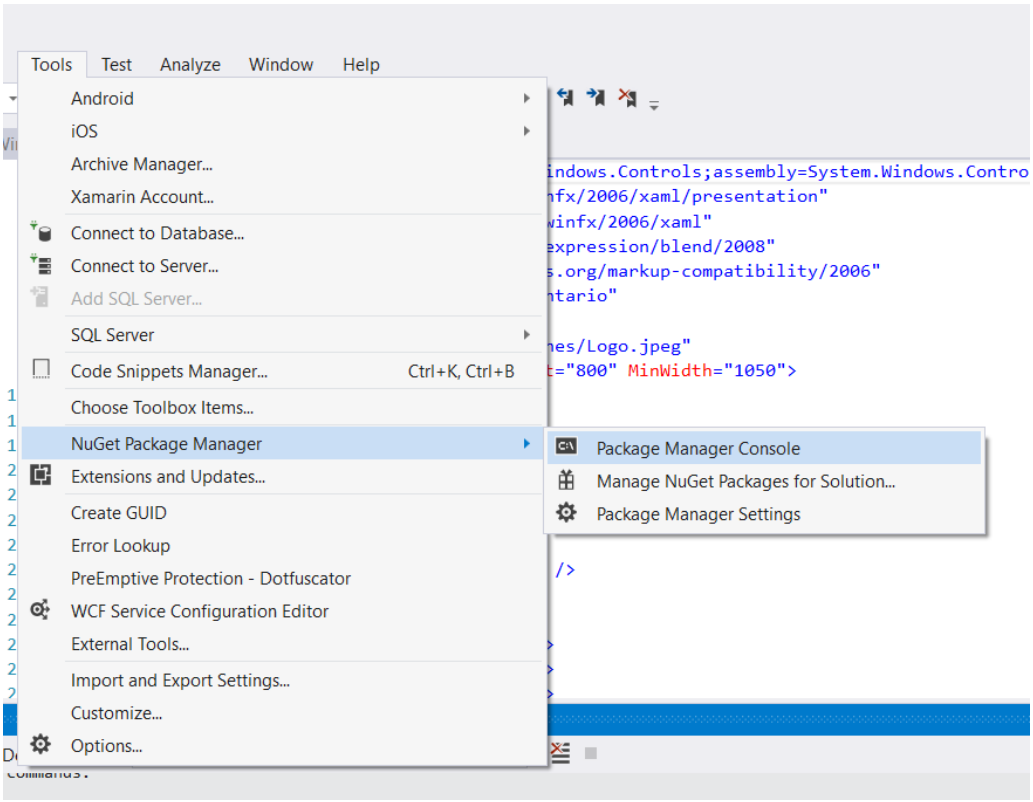
b) El siguiente es utilizar la utilidad NUGET (algo parecido al marketplace de Eclipse) que nos permite descargarnos utilidades y herramientas de los repositorios y luego instalarlos en nuestro proyecto.

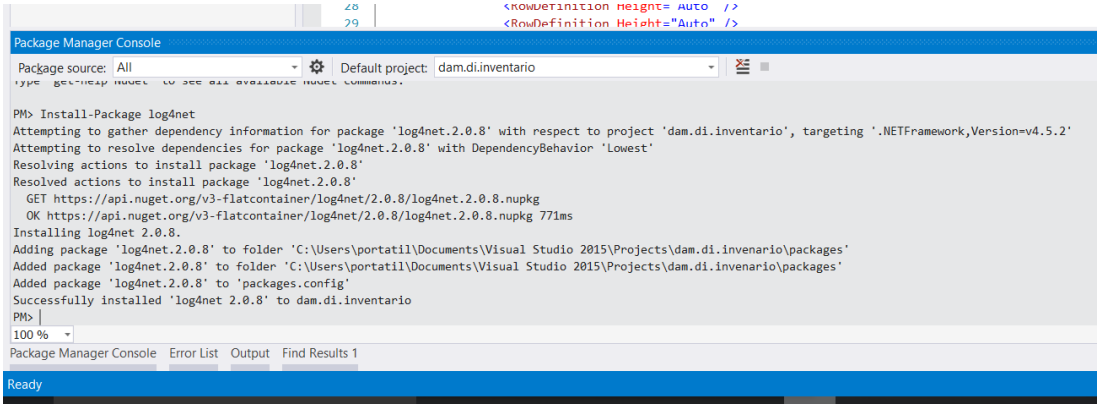


Nos muestra la siguiente pantalla:



Podemos hacerlo con una utilidad de comandos:



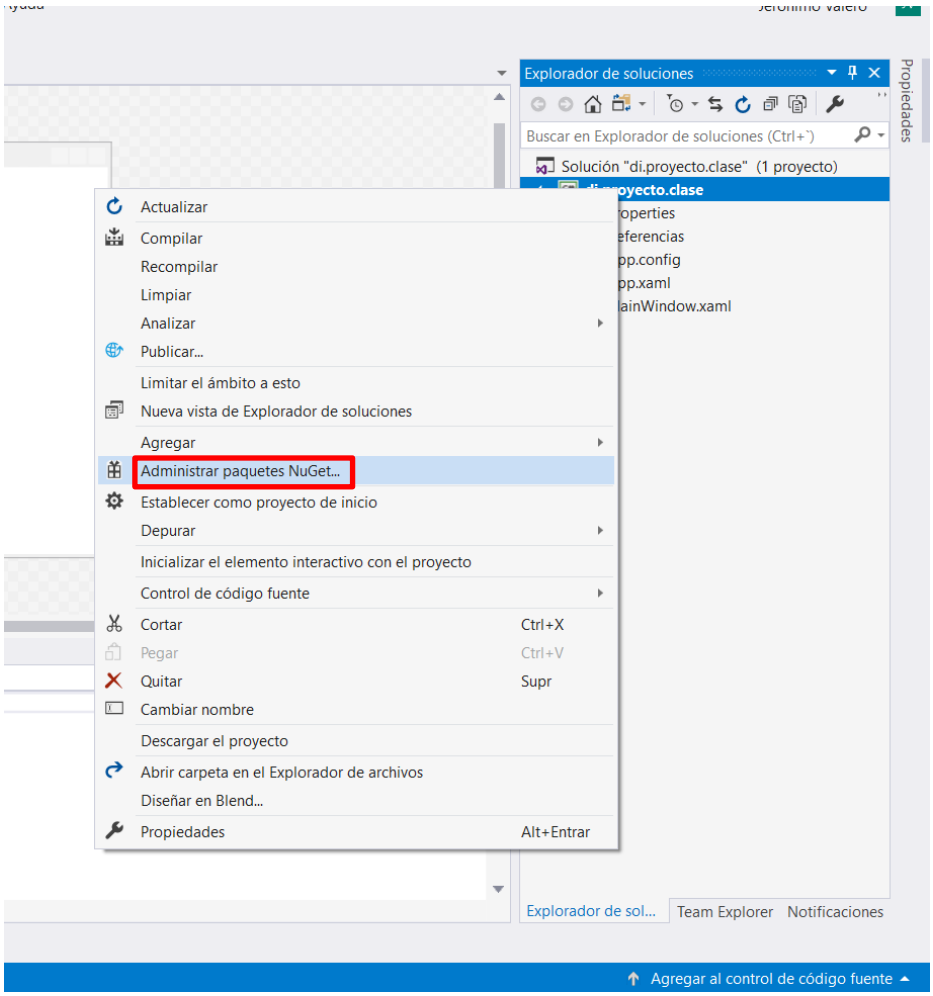


ANEXO II. MAHAPPS

Es una librería que podemos añadir a nuestro proyecto y que dispone de controles y estilos predefinidos que nos ayudan a mejorar nuestra aplicación.

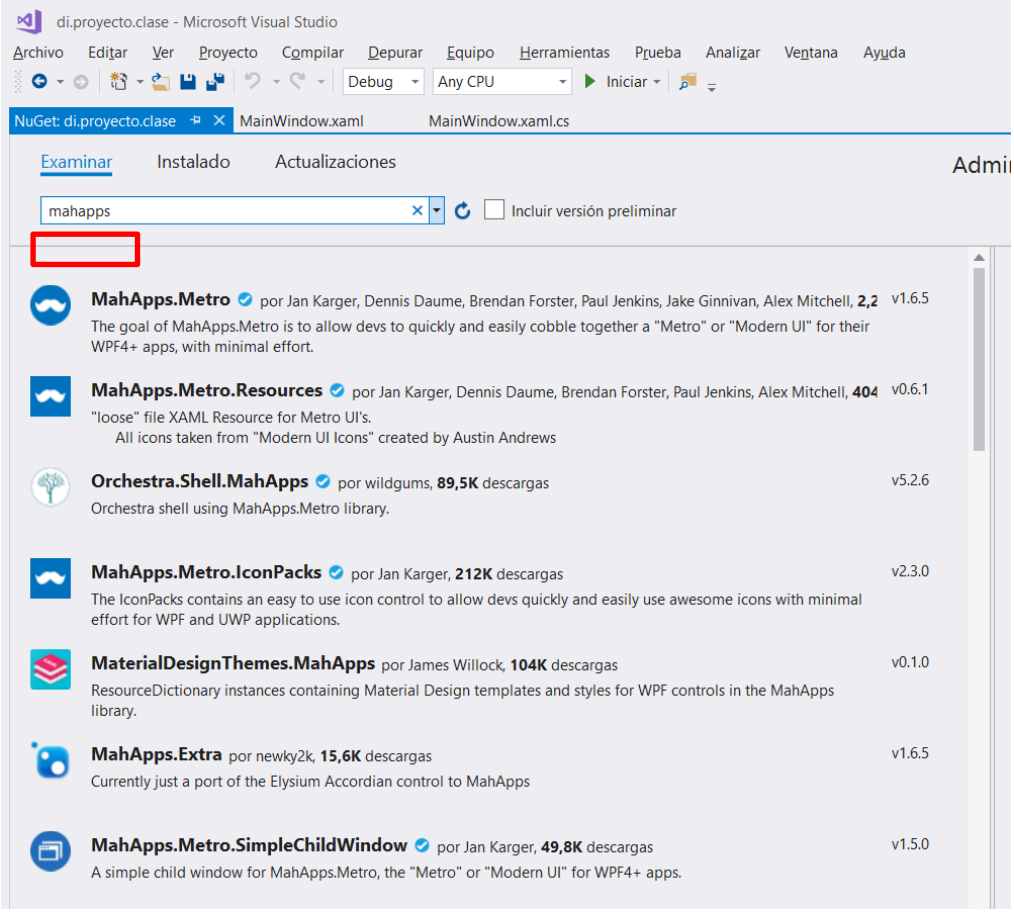
El sitio web en la que podemos encontrar documentación es https://mahapps.com/

Los pasos para su instalación son los siguientes:

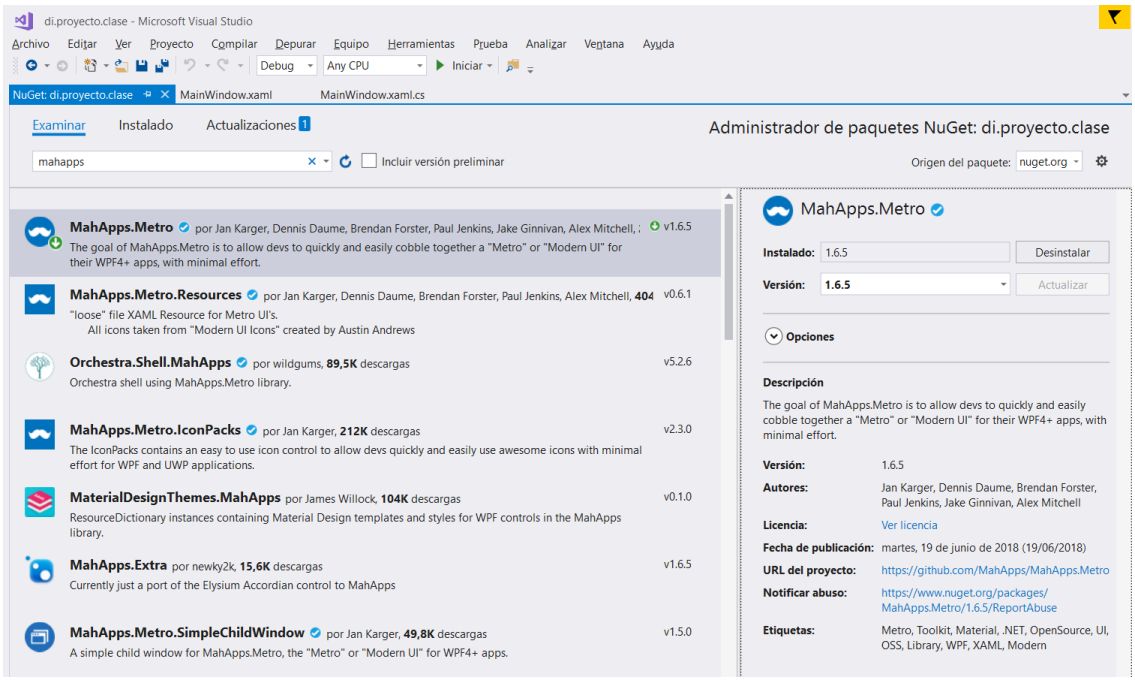


C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Primero debemos instalar el paquete, para ello seleccionamos nuestro proyecto y apretamos el botón derecho. Luego seleccionamos la opción de **Administrar paquetes NuGet …**

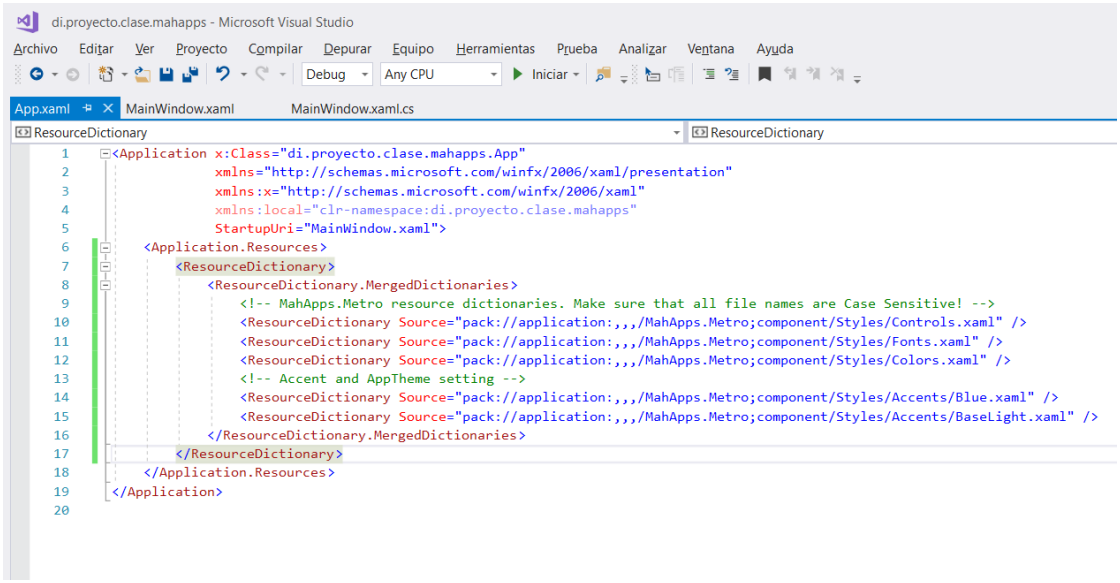


En esta pantalla seleccionamos la pestaña **Examinar** y buscamos el paquete que queremos instalar, en el caso que nos ocupa, **Mahapps**.



De la lista que nos muestra instalaremos los siguientes paquetes: ***MahApps.Metro, MahApps.Metro.Resources, MahApps.Metro.IconPacks*** y ***MahApps.Extra***.

Una vez instalados los paquetes debemos especificar los estilos propios de Mahapps en nuestro fichero **App.xaml**.

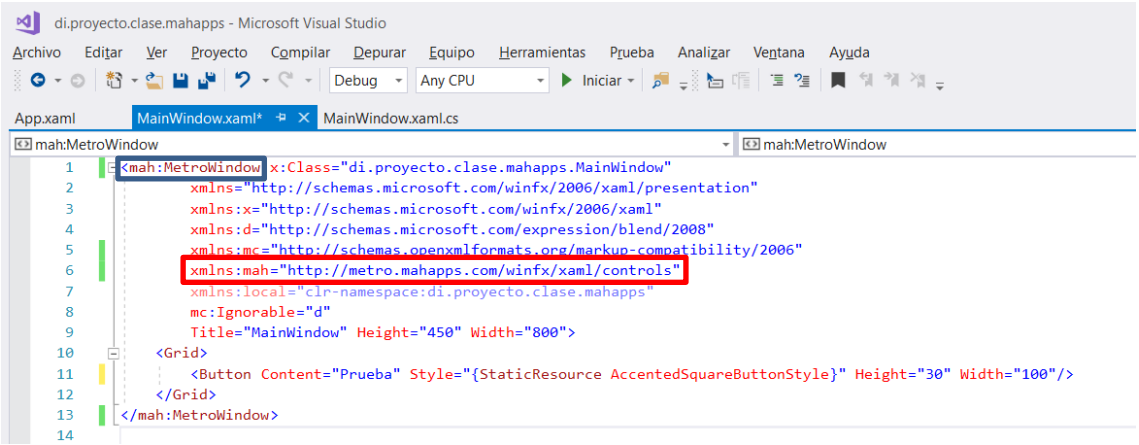


El código es el siguiente:

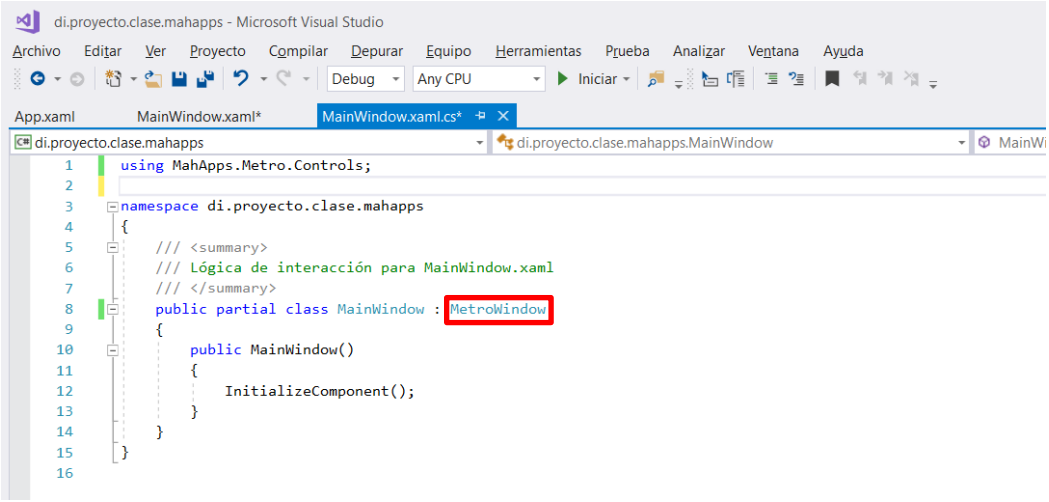
C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<ResourceDictionary>
      <ResourceDictionary.MergedDictionaries>
        <!-- MahApps.Metro resource dictionaries. Make sure that
all file names are Case Sensitive! -->
        <ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Contr
ols.xaml" />
        <ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Fonts
.xaml" />
        <ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Color
s.xaml" />
        <!-- Accent and AppTheme setting -->
        <ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Accen
ts/Blue.xaml" />
        <ResourceDictionary
Source="pack://application:,,,/MahApps.Metro;component/Styles/Accen
ts/BaseLight.xaml" />
      </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
```

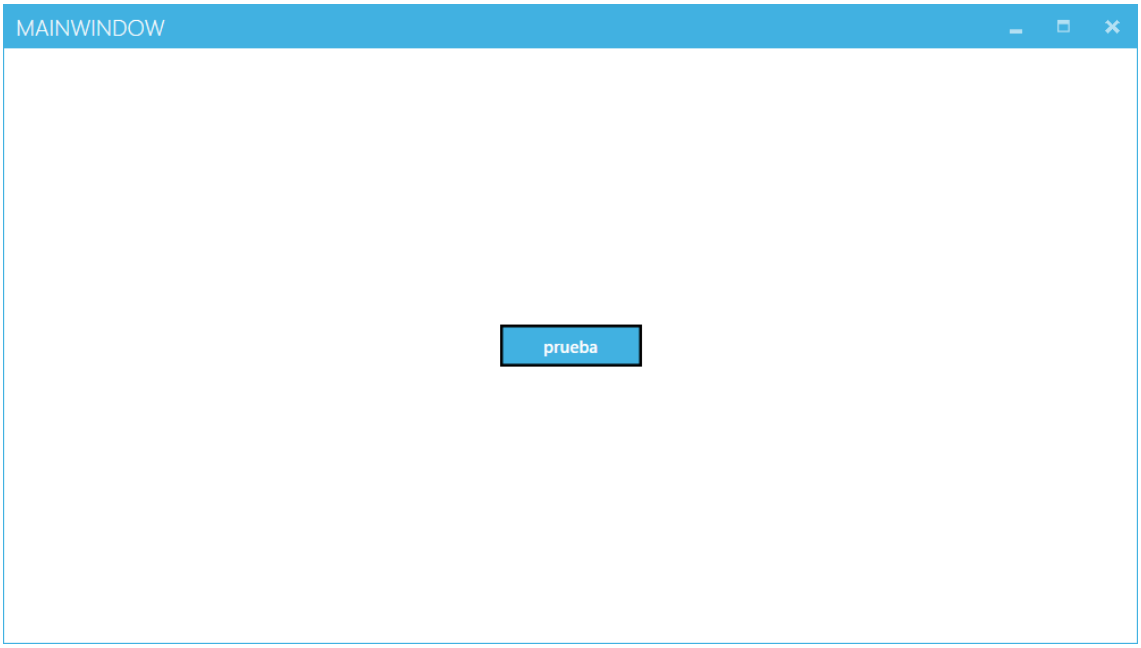
Por último debemos especificar la manera de utilizar los controles de Mahapps en la ventana.



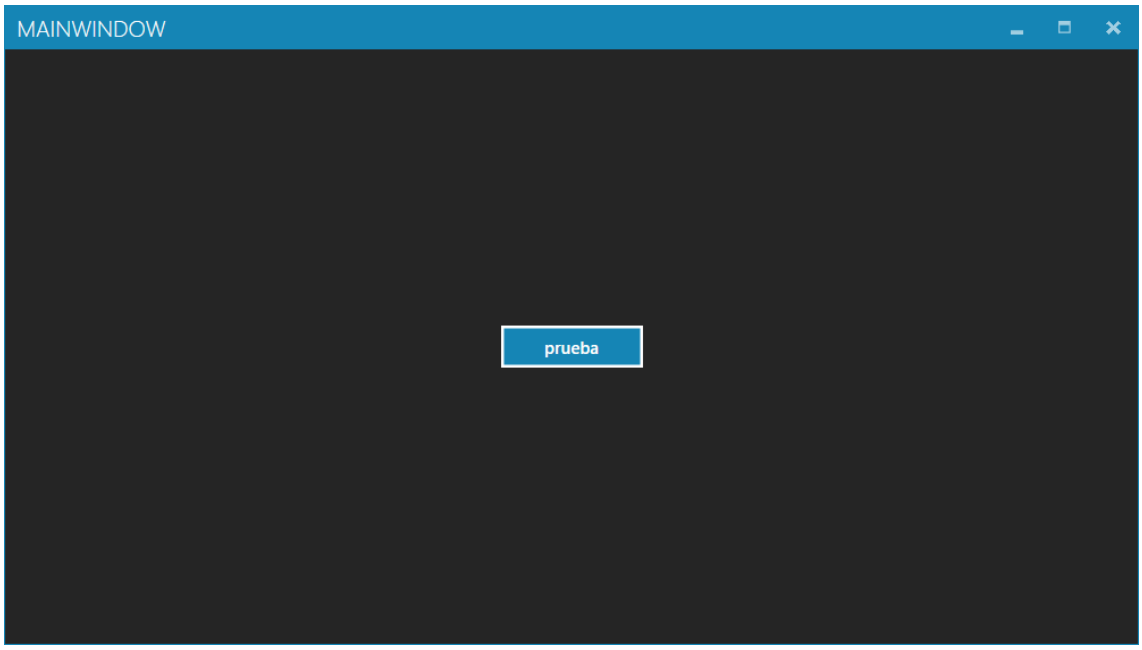
Añadimos la librería de etiquetas y luego ya podemos utilizarla. Además debemos cambiar la clase en nuestra clase espejo **MainWindow.xaml.cs**.



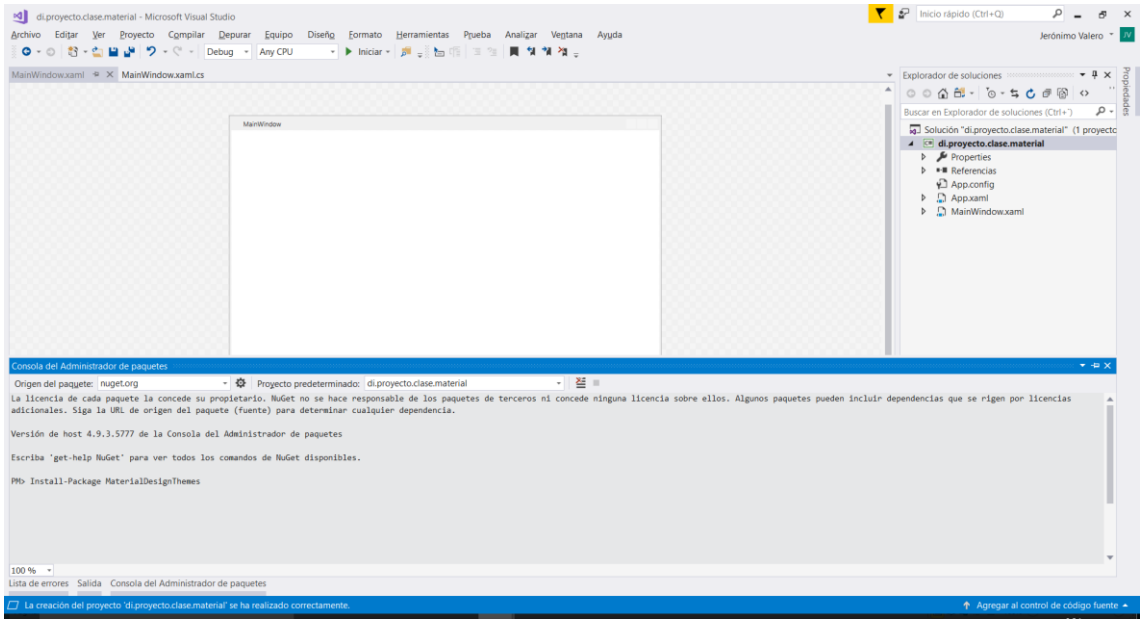
El resultado es el siguiente:



O cambiando el estilo en nuestro fichero **App.xaml**



Material Design



ANEXO III. Recursos

Los recursos nos permiten en WPF acceder a fuentes de datos que serán utilizados en nuestras aplicaciones, concretamente en nuestros ficheros XAML.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

En este apartado vamos a ver la forma de utilizar los recursos para la sincronización de datos. Podemos definir los recursos que utilizamos a nivel de **aplicación, de ventana o de layout**, para ello utilizaremos correspondiente seguida de *Resources*. Por ejemplo: ***<Window.Resources> </Window.Resources>***

También podemos distinguir entre **StaticResources** y **DynamicResources**. Los primeros son recursos que no cambian, es decir, son inicializados cuando cargamos el fichero xaml y luego permanecen invariables a los cambios puesto que no vuelven a refrescarse.

Mientras que los recursos dinámicos son revisados cada vez que se necesitan, y por tanto podemos modificarlos porque los cambios serán actulizados.

También podemos acceder a estos recursos desde el código mediante el método **FindResource**.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Bibliografía

http://www.wpftutorial.net/
http://www.wpf-tutorial.com/
http://www.tutorialspoint.com/wpf/
https://wpf.2000things.com/
http://www.c-sharpcorner.com/
https://msdn.microsoft.com/
http://www.blackwasp.co.uk/
http://miteshsureja.blogspot.com.es
https://www.codeproject.com